

第五章 行为学派

强化学习简介1



机器学习的分类

监督学习

输入: (x, y)

x 是数据, y 是标签

目的: 建立 x 到 y 的映射

$$y = f(x)$$

例子:



这是苹果

无监督学习

输入: x

x 是数据, 无标签

目的: 学习 x 的底层结构/特征

例子:



这些都是水果

强化学习

输入: 状态-动作空间

available action := f(state)

目的: 通过选择动作来最大化预期未来奖励

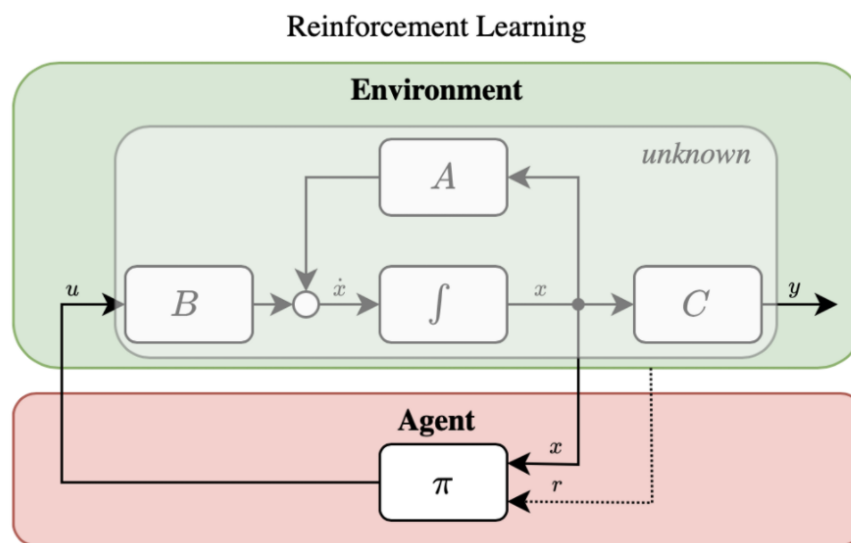
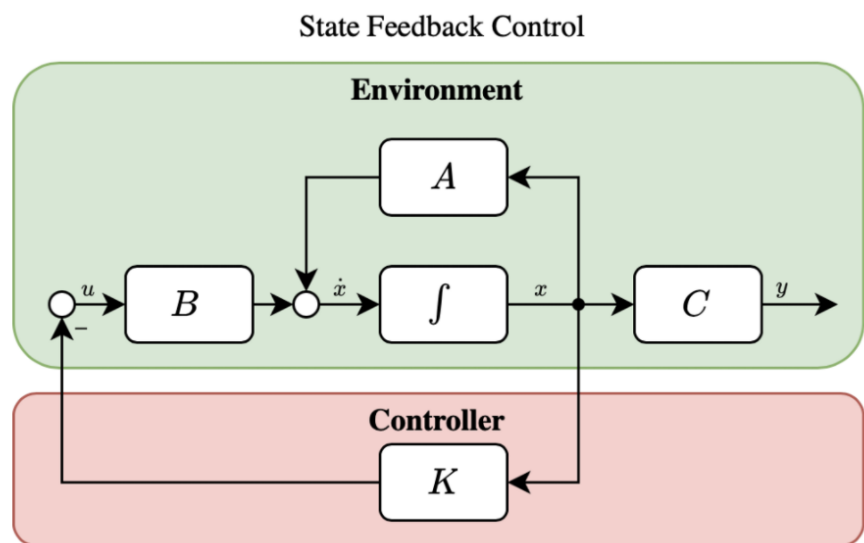
例子:



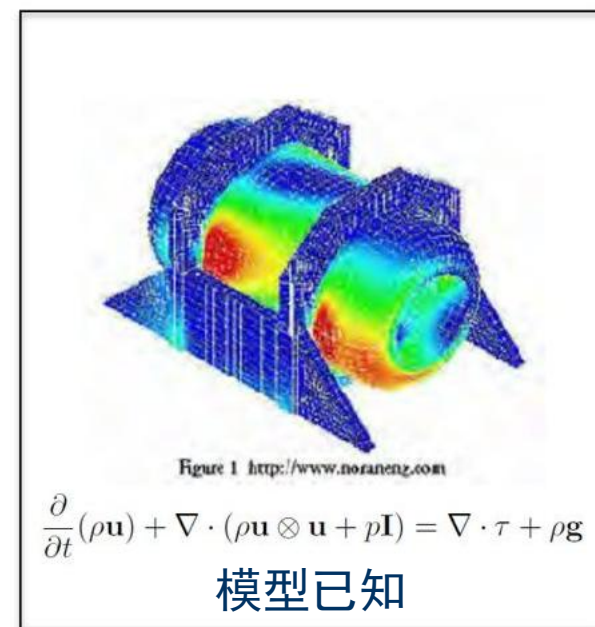
吃掉苹果来让自己(更好地)生存

强化学习与最优控制

- **最优控制**：以优化方法的形式框架，求取连续时间控制问题中的最优控制策略。
 - 假设模型已知（Model-based，如左图）
 - **动态规划**是求解最优控制问题的一种经典方法

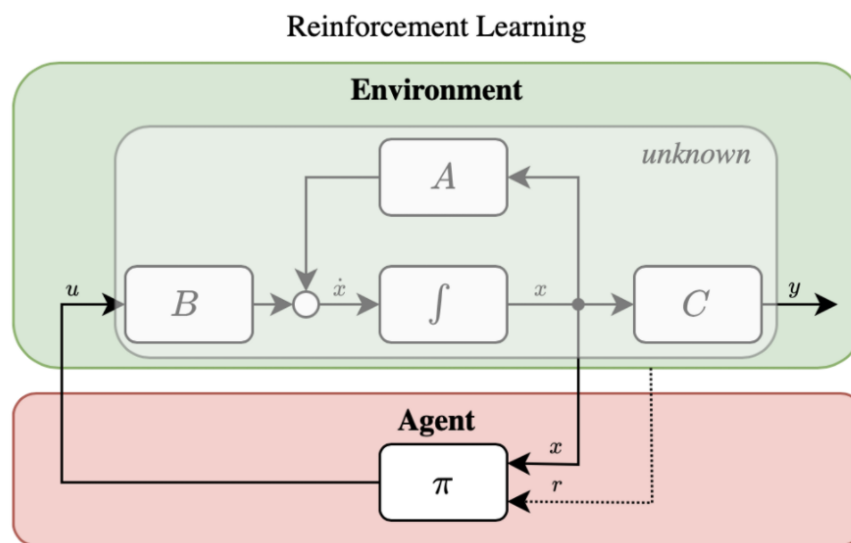
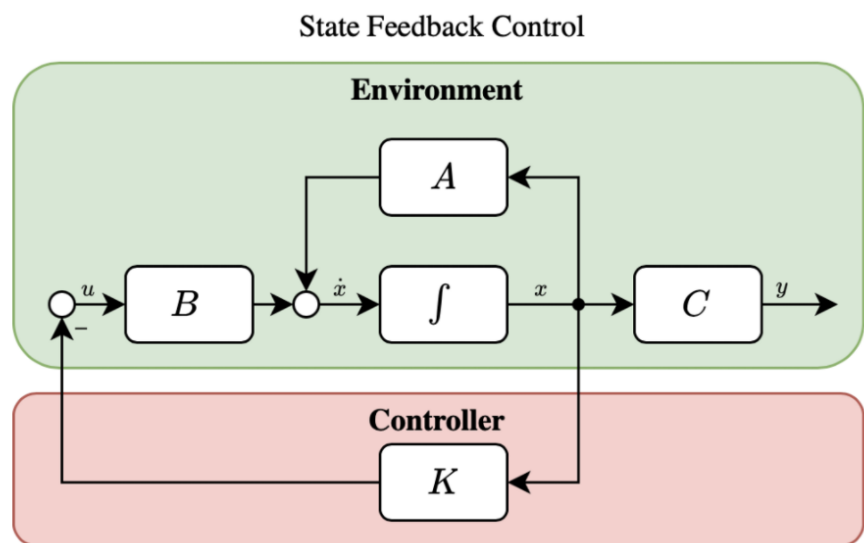


Classic control is more similar to reinforcement learning than one might think. However, the key distinction between them is, that in control we assume to know the underlying system dynamics, whereas in reinforcement we do not. |
(Image by author)

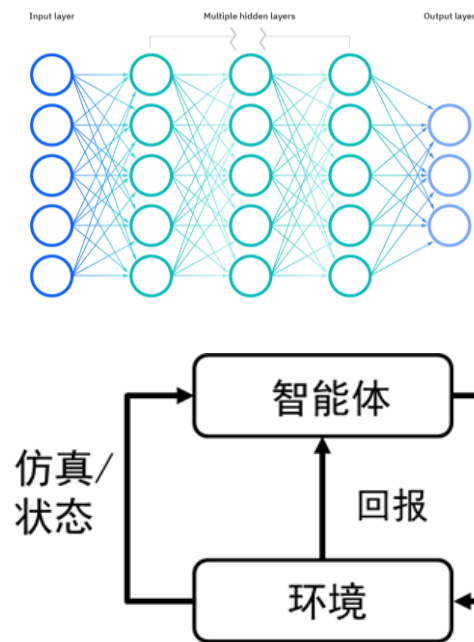


强化学习与最优控制

- **强化学习**：通过与未知和不确定（如随机）环境的直接交互（试错）学习一种行为策略，使长期的奖励总和（延迟奖励）最大化。
 - 模型通常未知（Model-free，如右图）



Classic control is more similar to reinforcement learning than one might think. However, the key distinction between them is, that in control we assume to know the underlying system dynamics, whereas in reinforcement we do not. |
(Image by author)



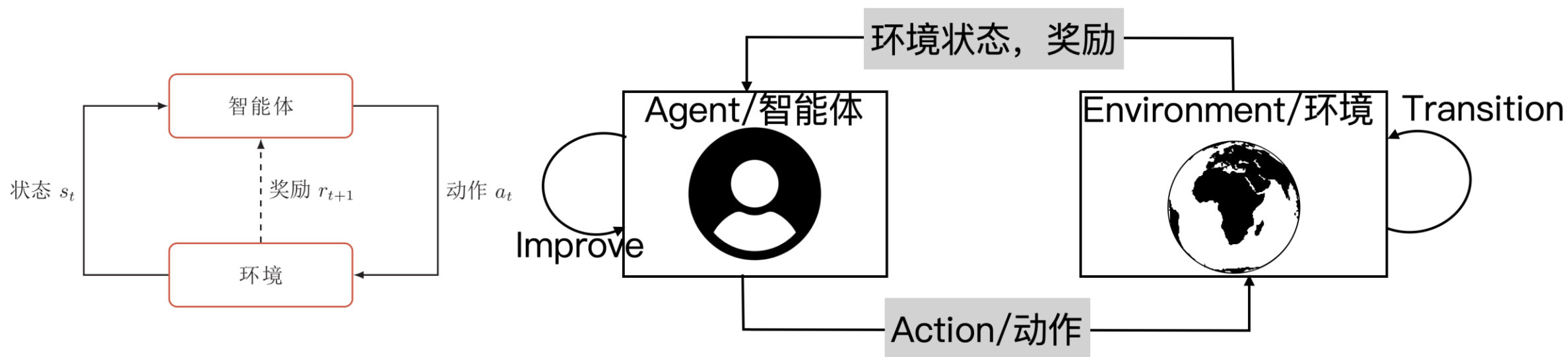
强化学习简介

- 强化学习的基本概念与历史
- 强化学习问题与基本方法
- 深度强化学习简介

强化学习简介

- 强化学习的基本概念与历史
- 强化学习问题与基本方法
- 深度强化学习简介

强化学习是什么



强化学习 (Reinforcement Learning, RL): 通过**智能体**与**环境**交互作用的一种**试错学习范式**

- 以最大化预期（未来累积）奖励为目的

强化学习的基本概念



假设你去遛狗想通过接飞盘

让你的小狗开心点儿

例子：小狗欢乐接飞盘

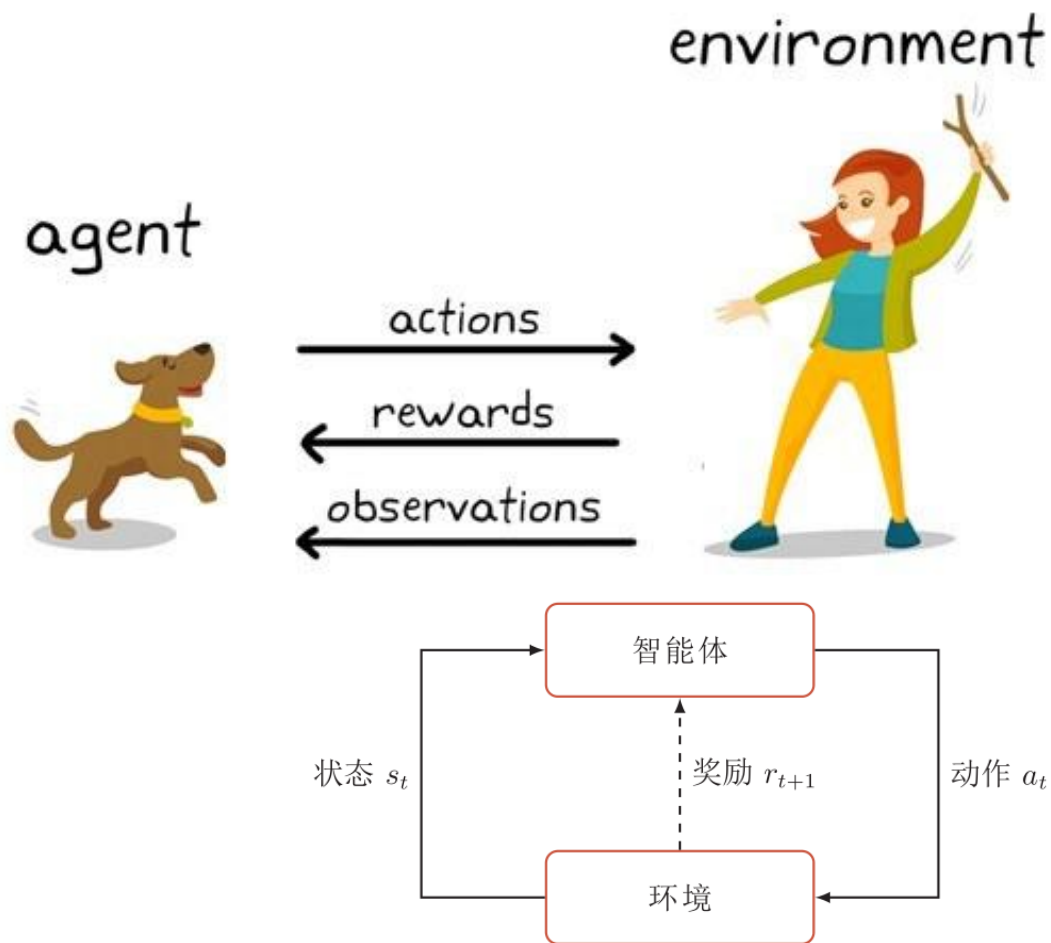
智能体：你的大脑中负责做决策的部分（简称为你）

环境：小狗、飞盘、草地、风速、噪音等

过程循环：

- (1) 你观察小狗的位置，草地情况等（观察）
- (2) 你丢出飞盘（动作）
- (3) 飞盘的位置随着你的作用力、风力等外作用改变位置、小狗奔跑改变位置和姿态、小狗没接到飞盘，不太开心（状态转移，释放奖励）
- (4) 你调整扔飞盘的方式，方便小狗接到（策略改进）
- (5) 回到(1)

强化学习的基本概念



• 要素:

⚡ 环境 Environment

⚡ 状态 State

⚡ 状态空间 State space

⚡ 观测 Observation

⚡ 动作 Action

⚡ 动作空间 Action space

⚡ 状态转移 Transition

⚡ 奖励 Reward

⚡ 智能体 Agent

⚡

强化学习的基本概念—环境

环境：除了智能体之外的一切

- 通常选择会影响决策的变量组合（状态）代表环境，状态所有可能的取值空间称为状态空间 / state space。例：小狗接飞盘的例子中，环境包括小狗、风、噪音、草地、小飞虫等
- 可能会有一些状态变量是智能体看不到的，智能体可以看到的状态变量称为观测, 可观测到的变量的取值空间称为观测空间 / observation space。例：小狗奔跑过程中被小飞虫叮了，导致没接到飞盘，但是主人没看到小飞虫（不可观测的变量）



动作与状态转移

环境：除了智能体之外的一切

- 在每一时刻，根据当前状态，环境会决定智能体可选择的动作有哪些，所有可能的动作组成**动作空间 / action space**。例：小狗接到飞盘，则**可选动作**为给好吃的，接不到飞盘时可选动作中就没有给好吃的这个选项了
- 环境在接收到智能体的动作后，会做出以下响应：
 - 通过其**状态转移函数 / transition function** 改变自身状态。
例：你选择扔出飞盘后，飞盘的位置，小狗的位置等都会改变



奖励与环境模型

环境：除了智能体之外的一切

- 在完成其自身的状态转移后，环境会通过其**奖励函数 / reward function** 释放出一个**瞬时**奖励信号。例：接到飞盘后小狗会表示开心，摇尾巴等
- **状态转移函数和奖励函数统称为环境模型 / model of the environment**
- Remark：奖励可以不来源于环境，而是智能体主观的激励信号，比如生物的自我激励、达成某种目标的体现形式等



另一种奖励



假设你想通过训练让小狗学会接飞盘

例子：训练小狗接飞盘

智能体：你的大脑中负责做决策的部分（简称为你）

环境：小狗、飞盘、草地、风速、噪音、好吃的等

过程循环：

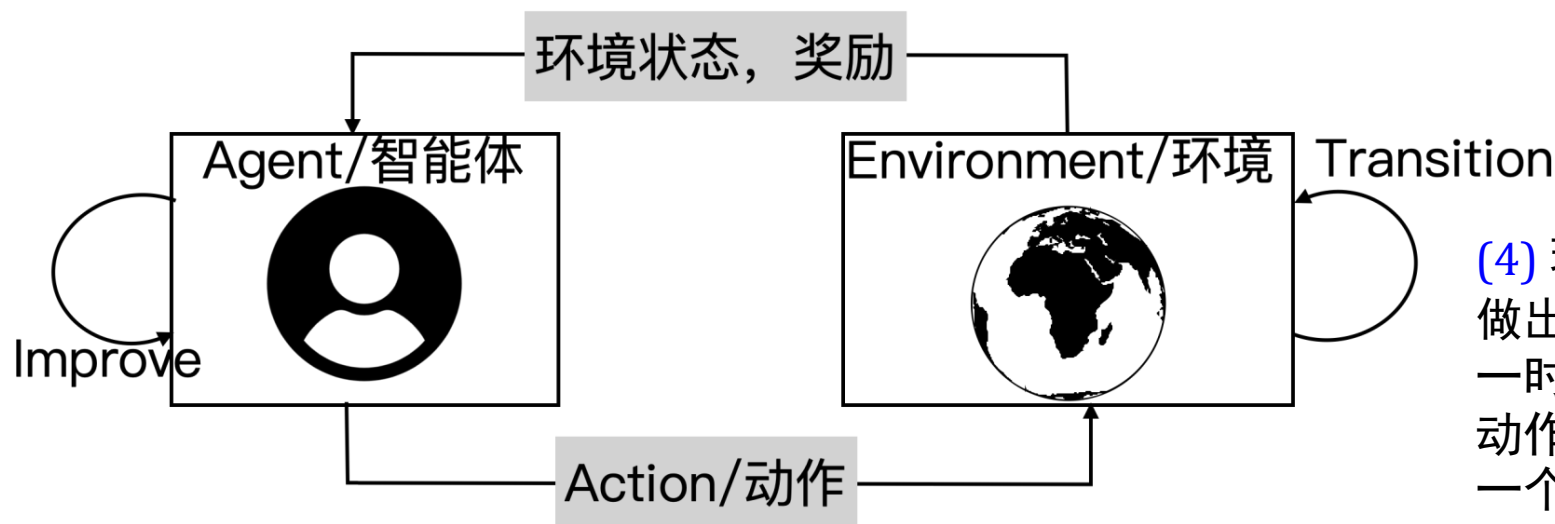
- (1) 你观察小狗的位置，草地情况等（观察）
- (2) 你丢出飞盘（动作）
- (3) 飞盘的位置随着你的作用力、风力等外作用改变位置、小狗奔跑改变位置和姿态、小狗(没)接到飞盘（状态转移，释放奖励）
- (4) 如果小狗接到，则给好吃的，没接到则继续扔飞盘（策略改进）
- (5) 回到(1)

强化学习是什么

循环：

(1) 智能体**观察**环境

(2) 智能体利用观测到的状态和奖励**改进**自己的 policy/策略



(4) 环境对智能体动作做出响应，根据自身上一时刻的状态和智能体动作（可能）**转移**到下一个状态。回到(1)

(3) 智能体选择并**执行**相应动作

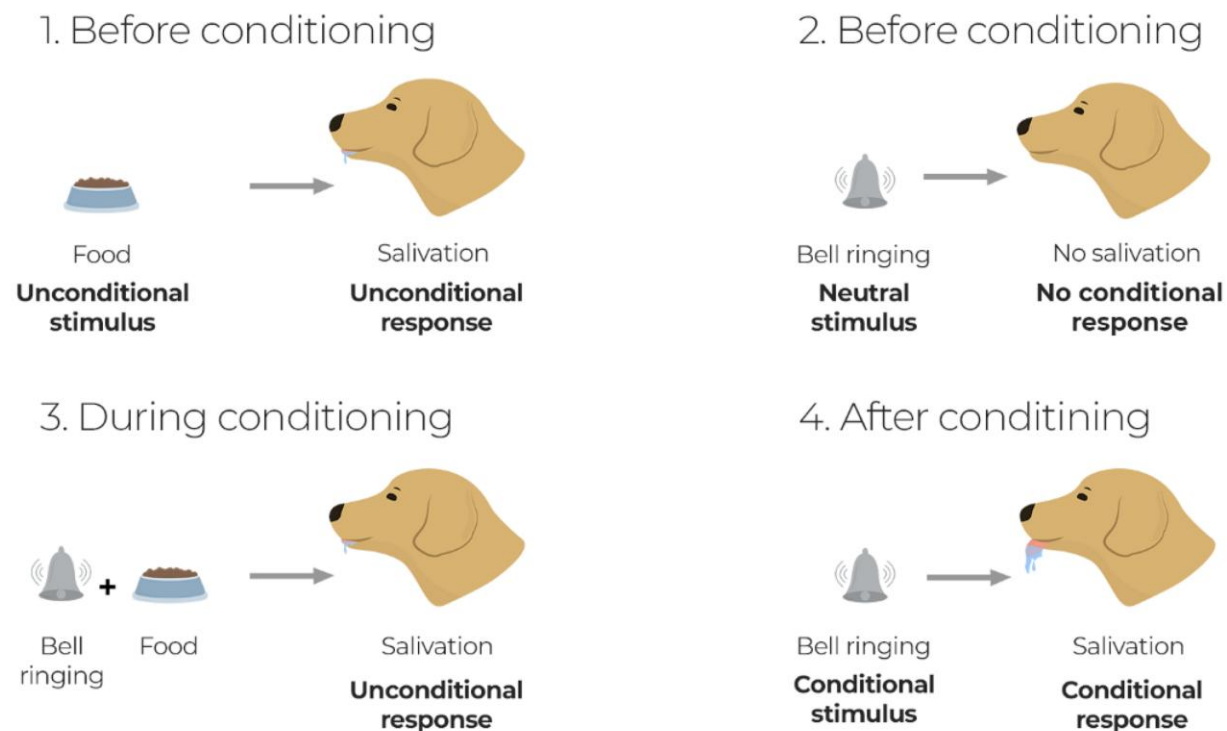
- 每个时刻 / time step，都进行如上图所示的循环。因此，RL是用来解决**序贯决策问题**的。
- 非常笼统地讲，RL包括**给定状态预测奖励的预测算法**和通过**试错来学习好的动作的控制算法**。

强化学习的由来

• 实验心理学的启发

1. 经典的动物条件反射

对应RL里的**预测算法**：
先学习到了环境状态的变化规律（铃铛之后会有吃的），然后基于这个规律在给定的状态下做出预测。



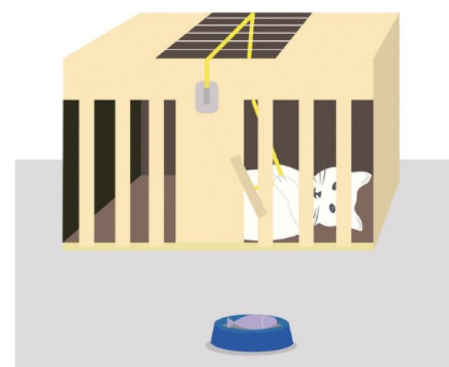
巴甫洛夫 I. Pavlov 实验设置，1927年

强化学习的由来

- 实验心理学的启发

- 2. 操作条件反射（或工具性条件反射）：

- 人类和动物学习行为以获得奖励(**obtain rewards**)和避免惩罚(**avoid punishments**)的过程 [斯金纳 B. F. Skinner, 1938]。
- 该领域先驱爱德华·李·桑代克（Edward Lee Thorndike）将小猫关进笼子里，让其自己学习打开门得到食物的过程。



操作条件反射先驱 E. Thorndike 小猫实验

强化学习的由来

• 实验心理学的启发

2. 操作条件反射（或工具性条件反射）：

对应RL中的**控制算法**，包含两个要点：

- 算法必须具有**选择性**，即需要尝试不同的动作，并根据动作的结果选择较好的动作
- 算法必须具有**关联性**，即需要讲动作和给定的状态关联起来（在特定的状态下选择某个动作）
- 得到奖励的动作被**强化了**

⚡ Remark: 强化指任何形式的反馈，既可以是正面的（奖励）也可以是负面的（惩罚）



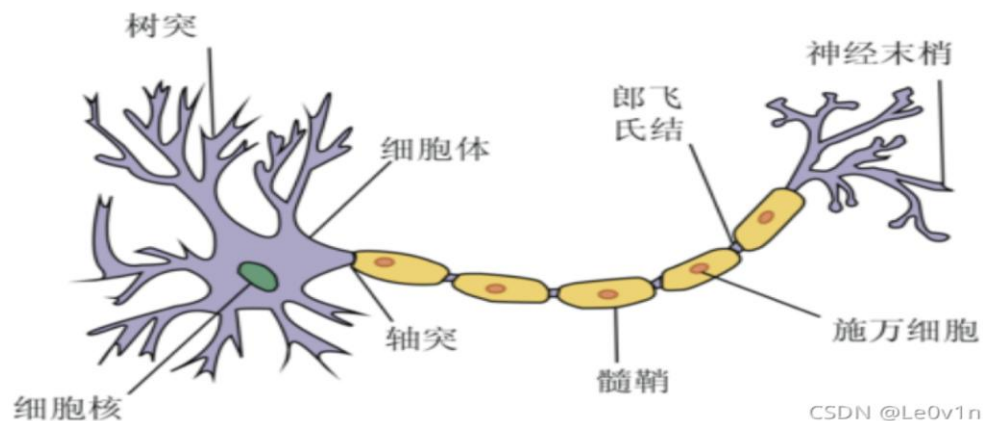
操作条件反射先驱 E. Thorndike 小猫实验

强化学习的由来

- 神经学启发

1. 赫布Hebbian学习:

- 通过共同激活神经元，来强化它们之间的突触权重，从而发展模型的形式。”如果先激活一个神经元，然后马上激活另一个神经元，它们就会连在一起”。[赫布 D. Hebb, 1961]。



Neurons that fire together, wire together.
— Donald Hebb

强化学习的由来

• 神经学启发

1. 赫布Hebbian学习:

- 通过共同激活神经元，来强化它们之间的突触权重，从而发展模型的形式。”如果先激活一个神经元，然后马上激活另一个神经元，它们就会连在一起”。[赫布 D. Hebb, 1961]。

例：一个人开始学习骑自行车的时候所有动作都是下意识的去做的，比如要先开锁，收起支撑架，手的动作，脚的动作，身体姿势等等。当学会骑车后这些动作就自然而然的出现了，他/她根本不用去下意识的思考接下来该怎么操作。也就是负责骑车决策的那部分神经元[让你获得最大利益，即不摔倒]之间的连接得到了强化/加强。



强化学习的由来

• 神经学启发

2. 多巴胺 / Dopamine模型:

- 首先由瑞典药理学家Arvid Carlsson（2000诺贝尔奖获得者）1957年发现。作为神经递质，帮助细胞传送**脉冲化学物质**，调控中枢神经系统的多种生理功能。
- 在学习过程中，Dopamine作为一种奖励**预测误差信号**来提高对**未来奖励的预测精度**，从而可以指导人/动物改进动作。

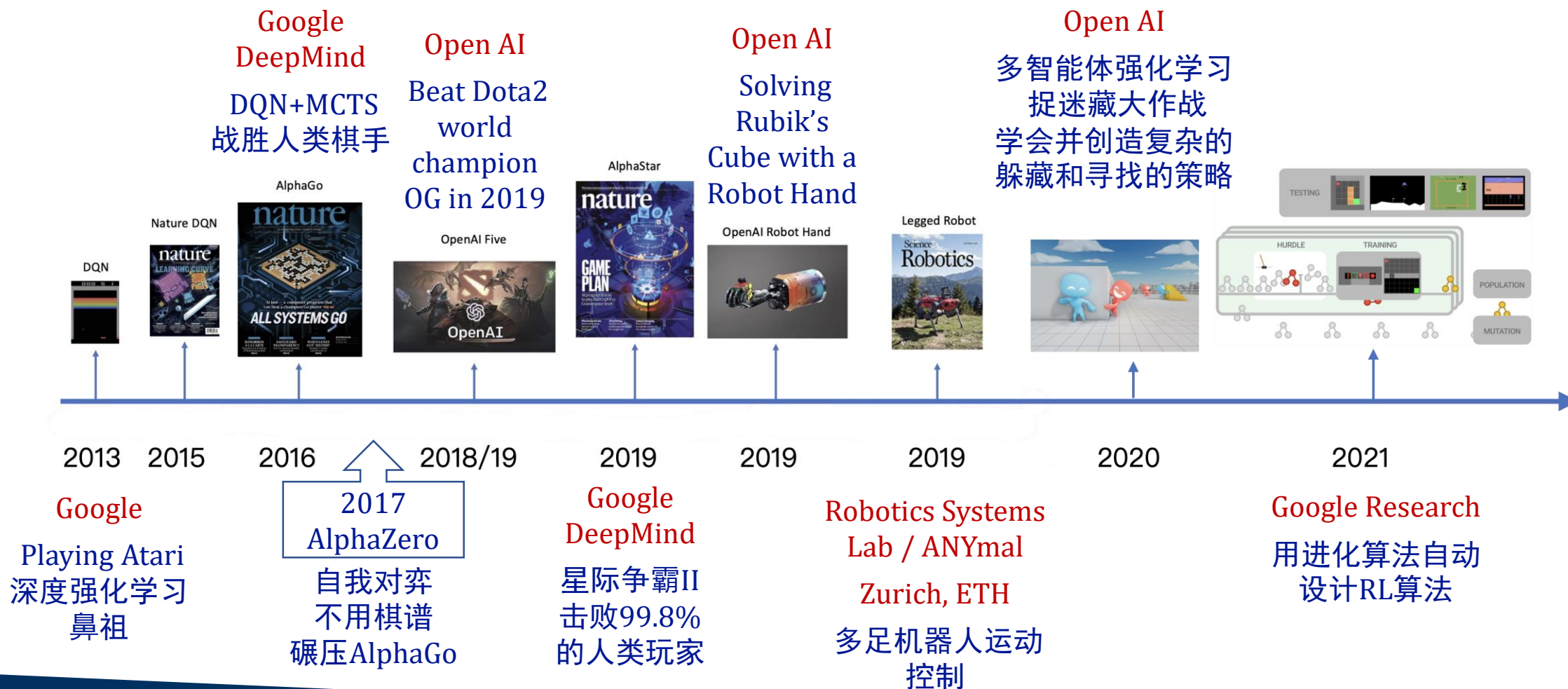


如果可以精准**预测某一动作对未来奖励的影响**，智能体就可以对全部允许采取的动作能够带来的奖励进行预测，进而选择带来最大奖励的动作。相当于RL中的**预测算法**。

强化学习的历史

- 计算机下棋程序 [香农, 1950 (论文1988)].
- 神经模拟强化系统理论 [明斯基, 1954].
- 利用跳棋游戏进行机器学习的研究 [Samuel, 1959].
- 试错 (井字棋) [米基, 1961].
- 自适应控制实验 (单连杆倒立摆) [米基 和 钱伯斯, 1968].
- 惩罚/奖励: 在自适应阈值系统中与批判者一起学习 (神经网络) [威德罗 等, 1973].
- 联想搜索网络。强化学习联想记忆 [Barto等, 1981].
- 强化学习中的时间分数分配” (时差学习, TD learning) [Sutton, 1984].
- 延迟奖励的学习 (Q学习) [Watkins, 1989].
- 时间差分法与TD-Gammon [Tesauro, 1995] 第一个利用神经网络的RL。

现代(深度)强化学习



现代(深度)强化学习

1. DQN: Playing Atari with Deep Reinforcement Learning <https://www.deepmind.com/publications/playing-atari-with-deep-reinforcement-learning>
2. Nature DQN: Human-level control through deep reinforcement learning <https://www.semanticscholar.org/paper/Human-level-control-through-deep-reinforcement-Mnih-Kavukcuoglu/e0e9a94c4a6ba219e768b4e59f72c18f0a22e23d>
3. AlphaGo: Mastering the game of Go with deep neural networks and tree search <https://www.nature.com/articles/nature16961>
4. OpenAI Five: Dota 2 with Large Scale Deep Reinforcement Learning <https://arxiv.org/abs/1912.06680>
5. AlphaStar: Grandmaster level in StarCraft II using multi-agent reinforcement learning <https://www.nature.com/articles/s41586-019-1724-z>
6. OpenAI robot hand: Solving Rubik's Cube with a Robot Hand <https://arxiv.org/abs/1910.07113>
7. Legged robot: Learning agile and dynamic motor skills for legged robots
https://www.science.org/doi/full/10.1126/scirobotics.aau5872?casa_token=sGRQXxPeIVYAAAAA%3AVl9WJaYBXvjbtT46KbLBWpv9N1X023dAMxEkWMwYLqp7hclgJv7u-tCHacv1RgpuKYwdMsga5V98NPA
8. Learning agile and dynamic motor skills for legged robots
https://www.science.org/doi/full/10.1126/scirobotics.aau5872?casa_token=sGRQXxPeIVYAAAAA%3AVl9WJaYBXvjbtT46KbLBWpv9N1X023dAMxEkWMwYLqp7hclgJv7u-tCHacv1RgpuKYwdMsga5V98NPA
9. Emergent Tool Use from Multi-Agent Interaction <https://arxiv.org/abs/1909.07528>
10. Evolving Reinforcement Learning Algorithms <https://ai.googleblog.com/2021/04/evolving-reinforcement-learning.html>

强化学习简介

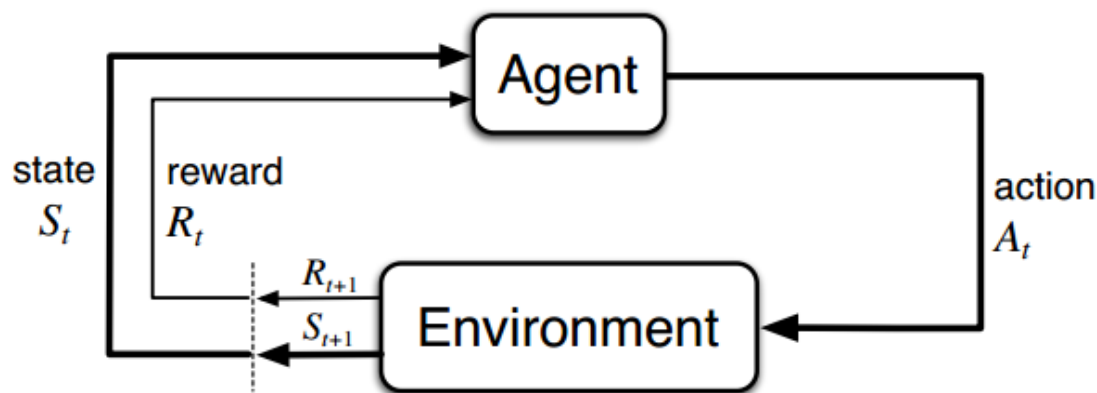
- 强化学习的基本概念与历史
- 强化学习问题与基本方法
- 深度强化学习简介

马尔可夫决策过程

马尔可夫决策过程（Markov Decision Process, MDP）是序贯决策（sequential decision）的数学模型，用于在系统状态具有马尔可夫性质的环境中模拟智能体可实现的随机性策略与回报。MDP的得名来自于俄国数学家安德雷·马尔可夫。

MDP基于一组交互对象，即智能体和环境进行构建，所具有的要素包括状态、动作、策略和奖励。在MDP的模拟中，智能体会感知当前的系统状态，按策略对环境实施动作，从而改变环境的状态并得到奖励，奖励随时间的积累被称为回报。

马尔可夫过程 (Markov Process, MP)



Agent每完成一个action，就会从environment得到反馈——state和reward。
总结起来说，action就是一个任务中agent做出的选择；state是做出选择的基础；reward是评价这个选择做的好不好的基础。

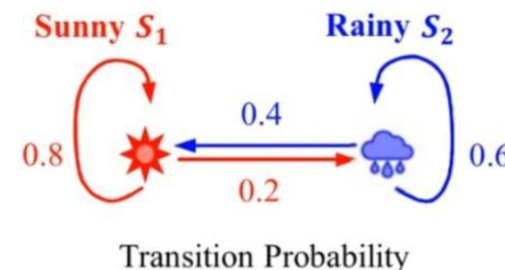
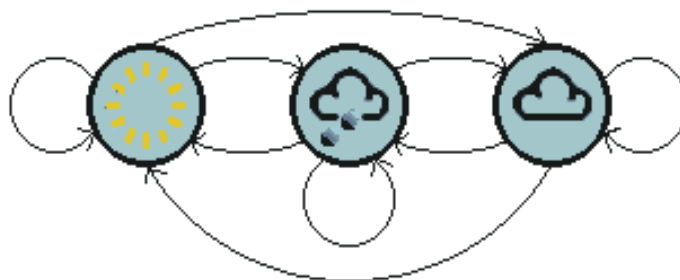
马尔可夫过程 (Markov Process, MP)

我们说一个state若满足 $P[s_{t+1}|s_t] = P[s_{t+1}|s_1, \dots, s_t]$, 则其具有马尔可夫性, 定义中可以看到, 当前状态state完全包含了历史中的所有信息, 一旦当前状态已知, 历史信息将会被抛弃。马尔科夫过程是无记忆的随机过程, 即随机状态序列具有马尔可夫属性。

一个马尔科夫过程可以由一个元组组成

- S 为（有限）的状态（state）集；
- P 为状态转移矩阵。所谓状态转移矩阵就是描述了一个状态到另一个状态发生的概率, 所以矩阵每一行元素之和为1。

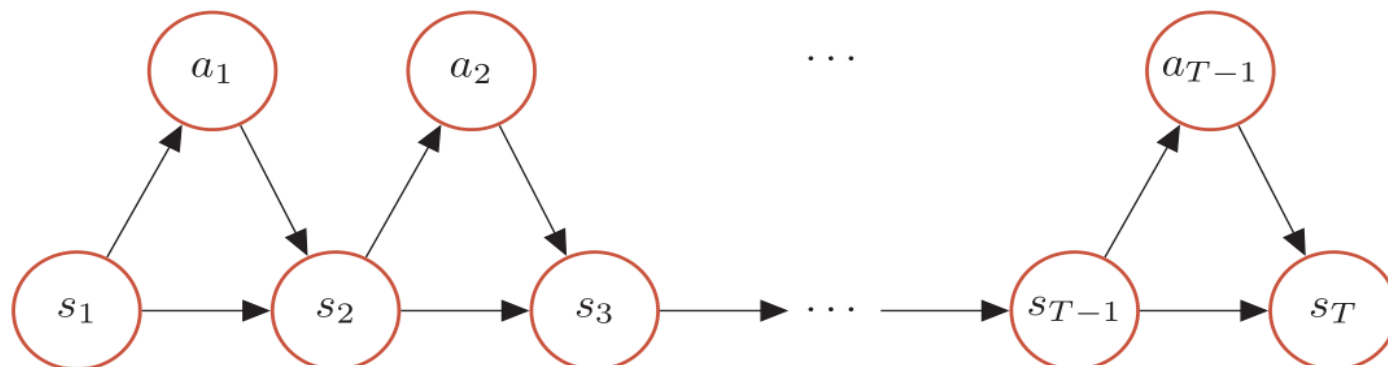
- 马尔可夫过程:
二元组: $\langle S, P \rangle$



	S_1	S_2
S_1	0.8	0.2
S_2	0.4	0.6

马尔可夫决策过程 (MDP)

- 马尔可夫决策过程：
五元组： $\langle S, A, P, R, \gamma \rangle$
无记忆性/无后效性



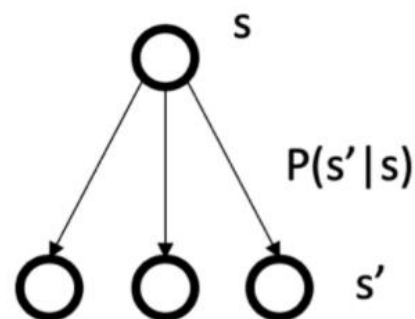
$s_0, a_0, s_1, r_1, a_1, \dots, s_{t-1}, r_{t-1}, a_{t-1}, s_t, r_t, \dots,$

- S 为（有限）的状态（state）集；
- A 为有限的动作集；
- P 为状态转移矩阵。所谓状态转移矩阵就是描述了一个状态到另一个状态发生的概率，所以矩阵每一行元素之和为1，
- R 为奖励函数（reward function），
- γ 为折扣因子，范围在 $[0,1]$ 之间，越大，说明agent看得越“远”。

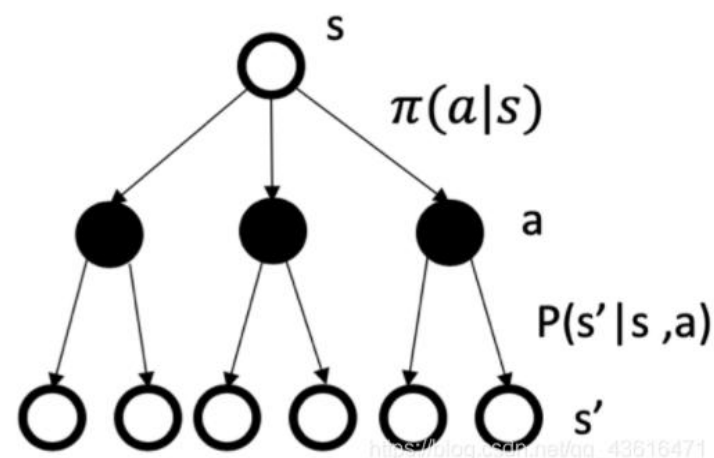
$$p(s_{t+1}|s_t, a_t, \dots, s_0, a_0) = p(s_{t+1}|s_t, a_t),$$

马尔可夫决策过程 (MDP)

- 马尔可夫过程：
二元组： $\langle S, P \rangle$

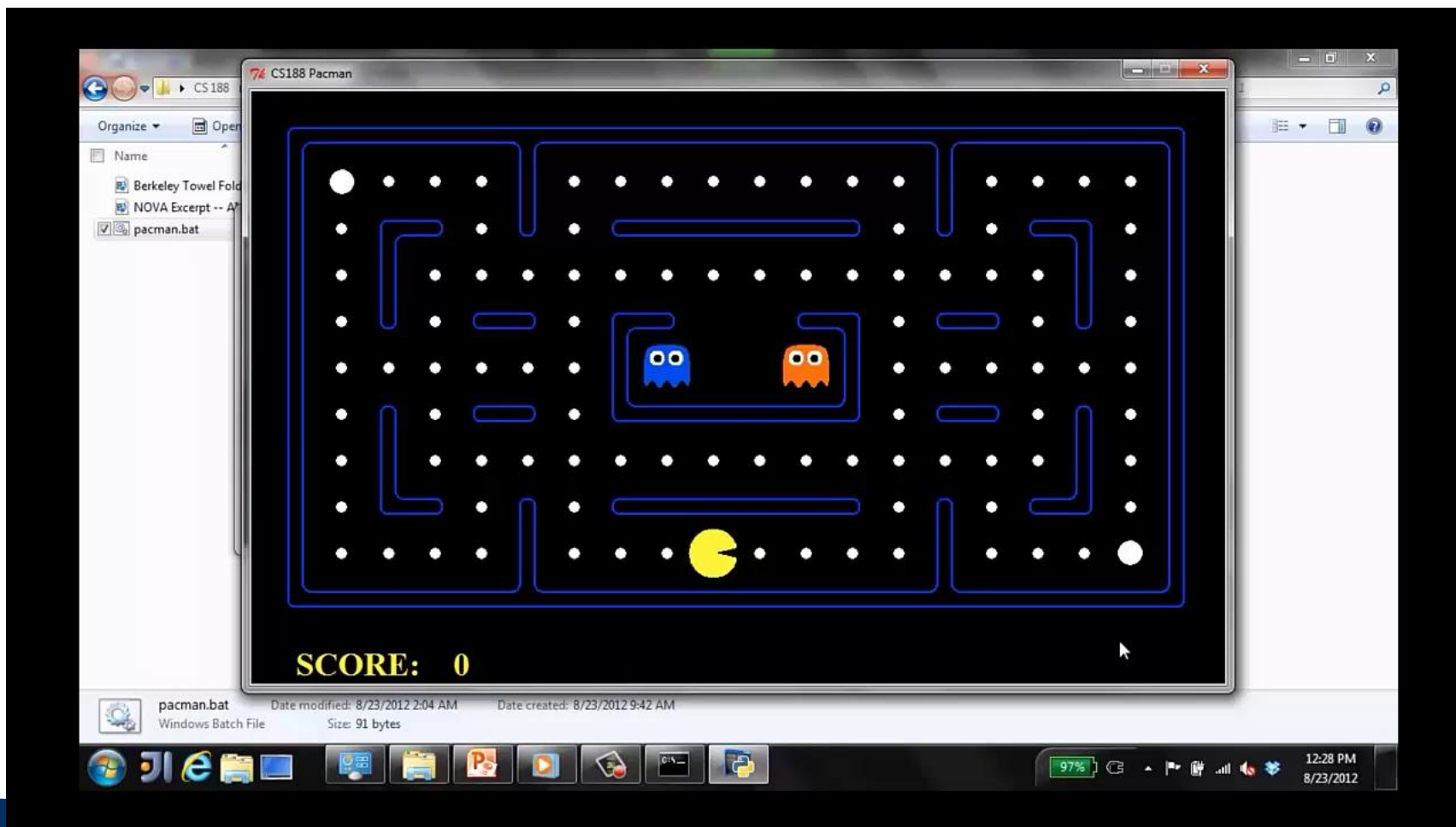


- 马尔可夫决策过程：
五元组： $\langle S, A, P, R, \gamma \rangle$



马尔可夫性质是概率论中的一个重要概念，它描述了一个随机过程的**未来状态仅依赖于当前状态**，而与过去的状态无关。

游戏例子—吃豆人



用规划的方式求解

↓ 游戏运作方式**已知**（有模型）

- 相当于可以向一个完美模型可供查询

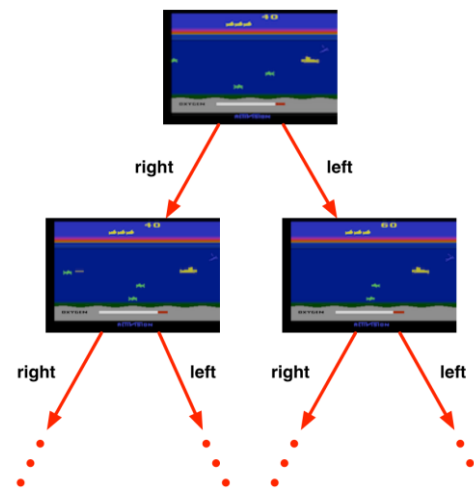
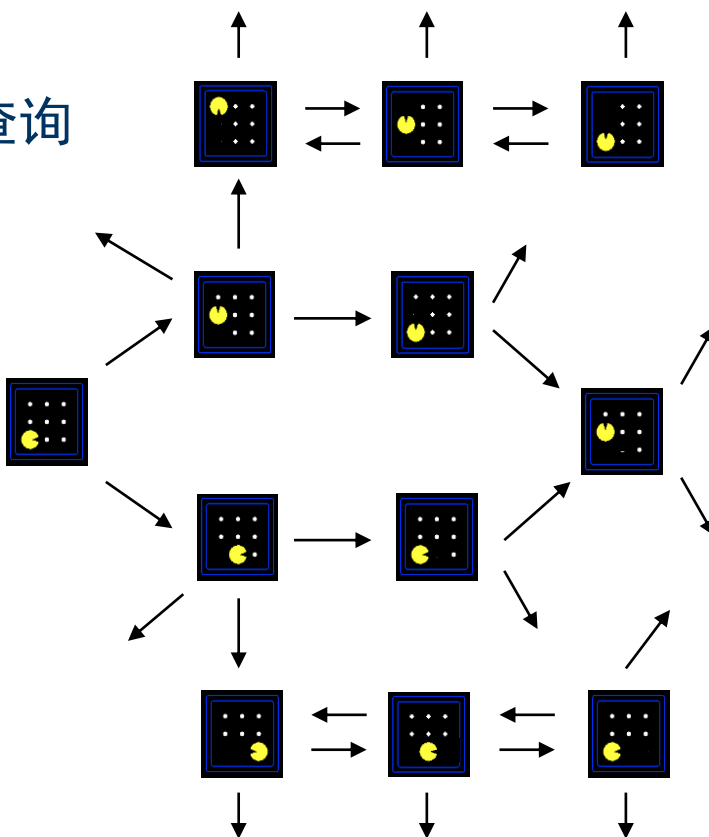
↓ 如果在**状态s**下采取**动作a**

- 下一个状态是？
- 分数会变成？

⚡ 统统已知！

↓ 能够在执行前找到最优策略

- 例：树搜索



搜索树

一个简单的例子



冰冻湖 (Frozen Lake, FL)



FL游戏案例

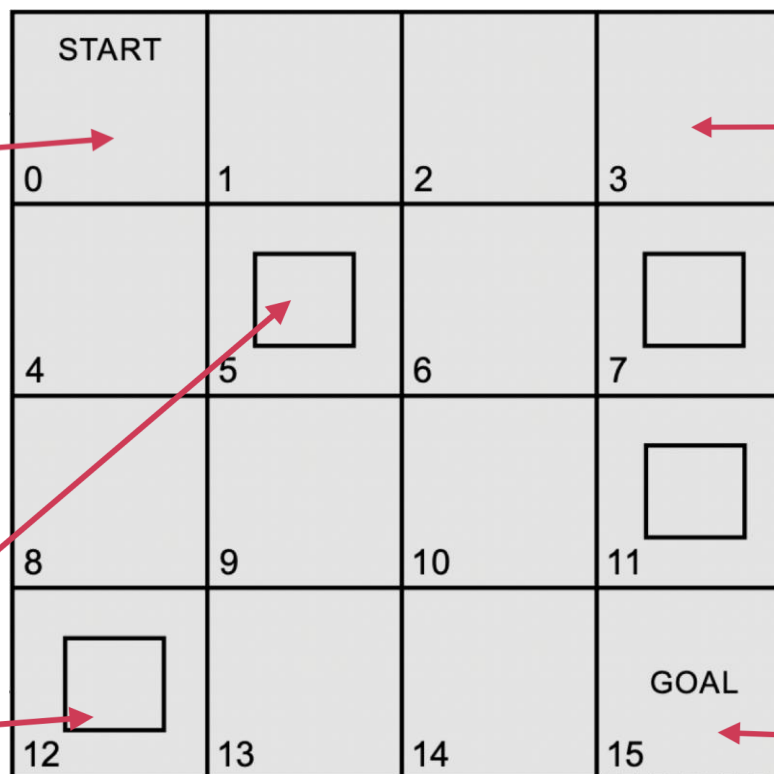
The **frozen lake** (FL) problem

注意：这是一个回合游戏

每次游戏开始时
Agent的位置

湖面很滑，Agent
可能会被滑到它
意想不到的地方

湖面有破洞，Agent
如果到这些地方的话，
那就被淹死了！

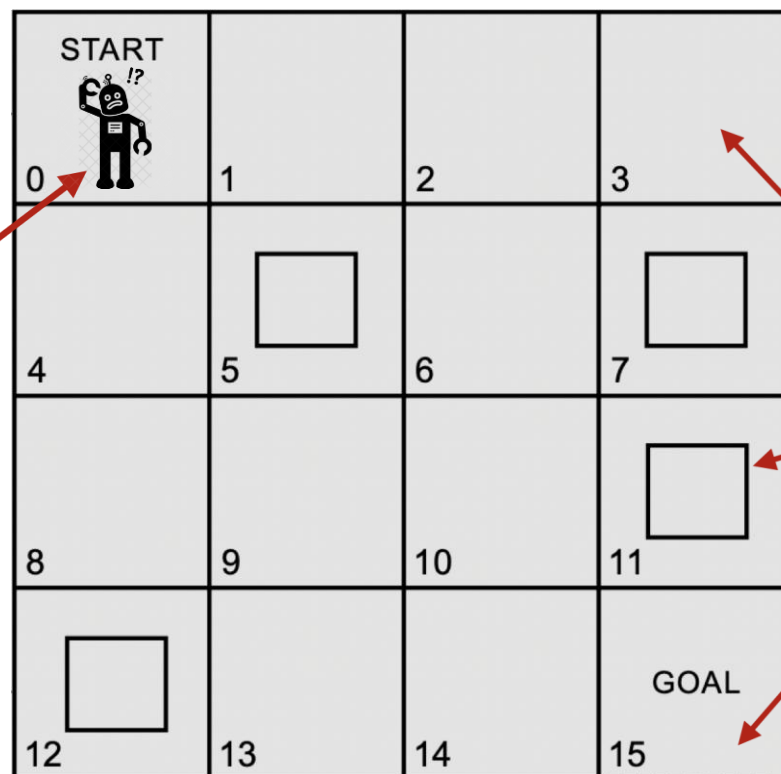


目标位置，
Agent走到这里
会收到+1的奖励

FL游戏案例

• 基本组成

Agent: 决定怎么从开始位置走到目标位置



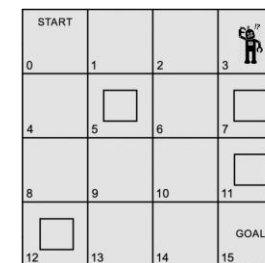
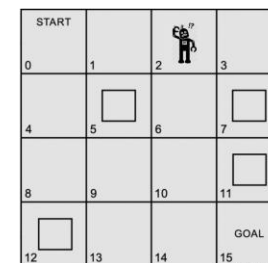
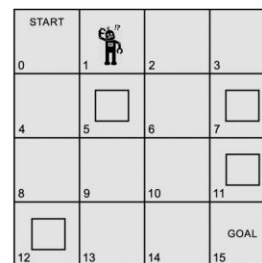
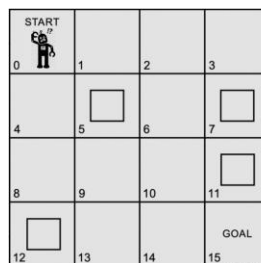
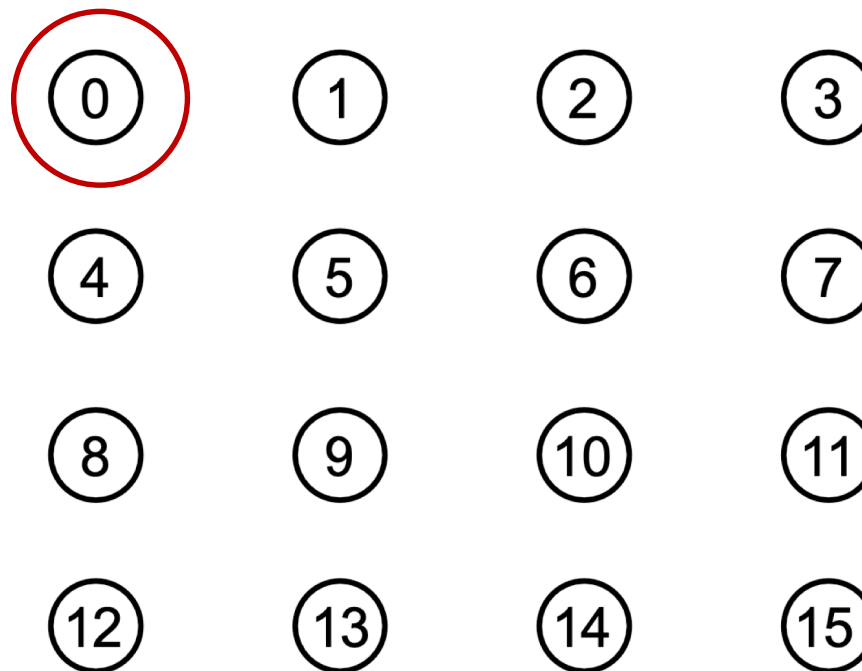
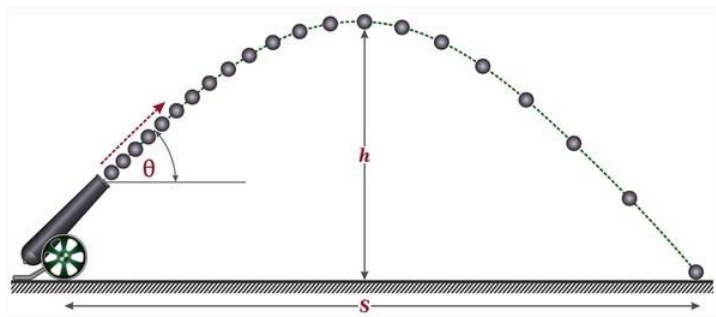
Environment: 光滑冰面，湖面上的洞，目标位置、湖面大小等。

FL游戏案例

怎么表示环境？

State: Agent在环境中的位置

选择状态可以有多种选法，但是这些状态需要满足**Markov性质**



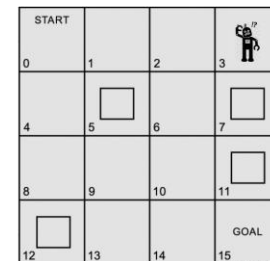
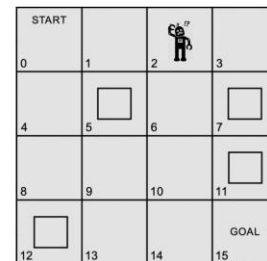
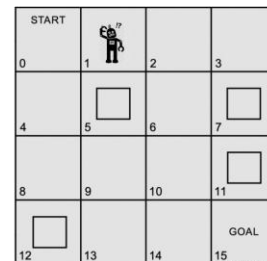
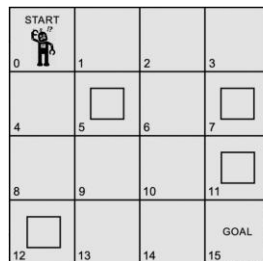
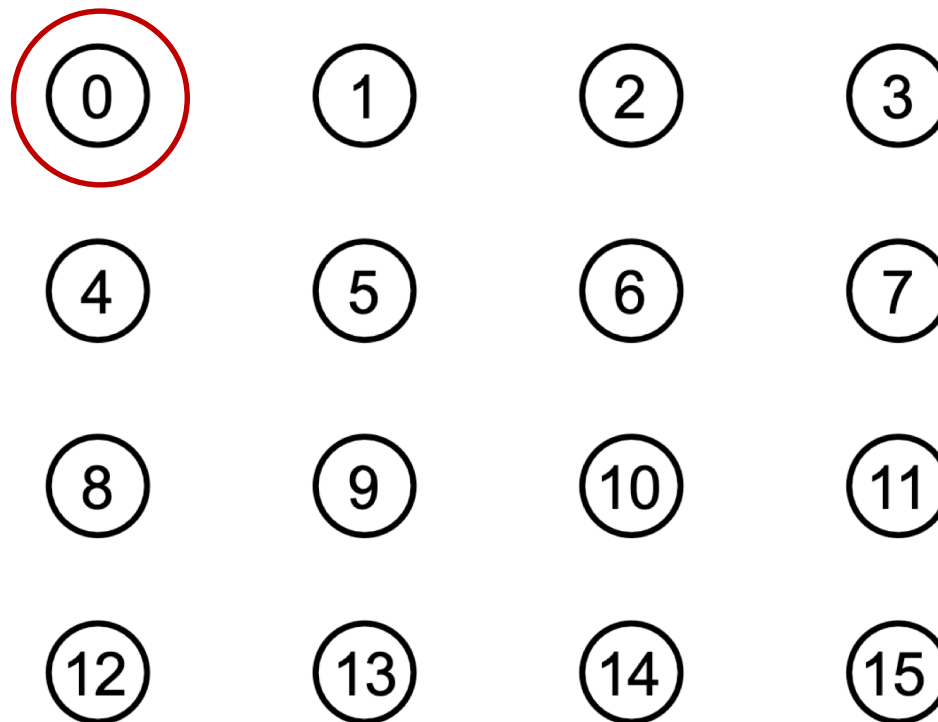
MDP是在环境中模拟智能体的**随机性策略（policy）与回报**的数学模型，且环境的状态具有**马尔可夫性质**。

FL游戏案例

怎么表示环境？

State space: 所有状态的组合

[[0], [1], [2], [3],
[4], [5], [6], [7],
[8], [9], [10], [11],
[12], [13], [14], [15]]



FL游戏案例

怎么表示环境?

Transition: 环境响应Agent动作的动态特性

Transition function:

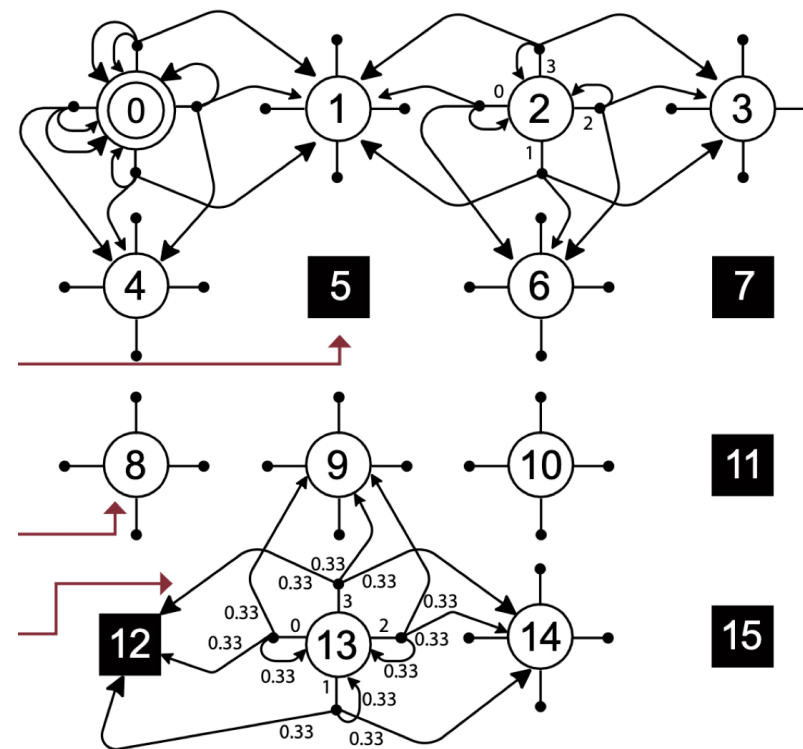
$$p(s'|s, a) = P(S_t = s' | S_{t-1} = s, A_{t-1} = a)$$

由概率的性质可知

$$\sum_{s' \in S} p(s'|s, a) = 1, \forall s \in S, \forall a \in A(s)$$

终态所有动作
都无用，还是
保持在终态

1/3的概率朝预期
方向移动，2/3概
率朝预期方向垂
直的方向移动



FL游戏案例

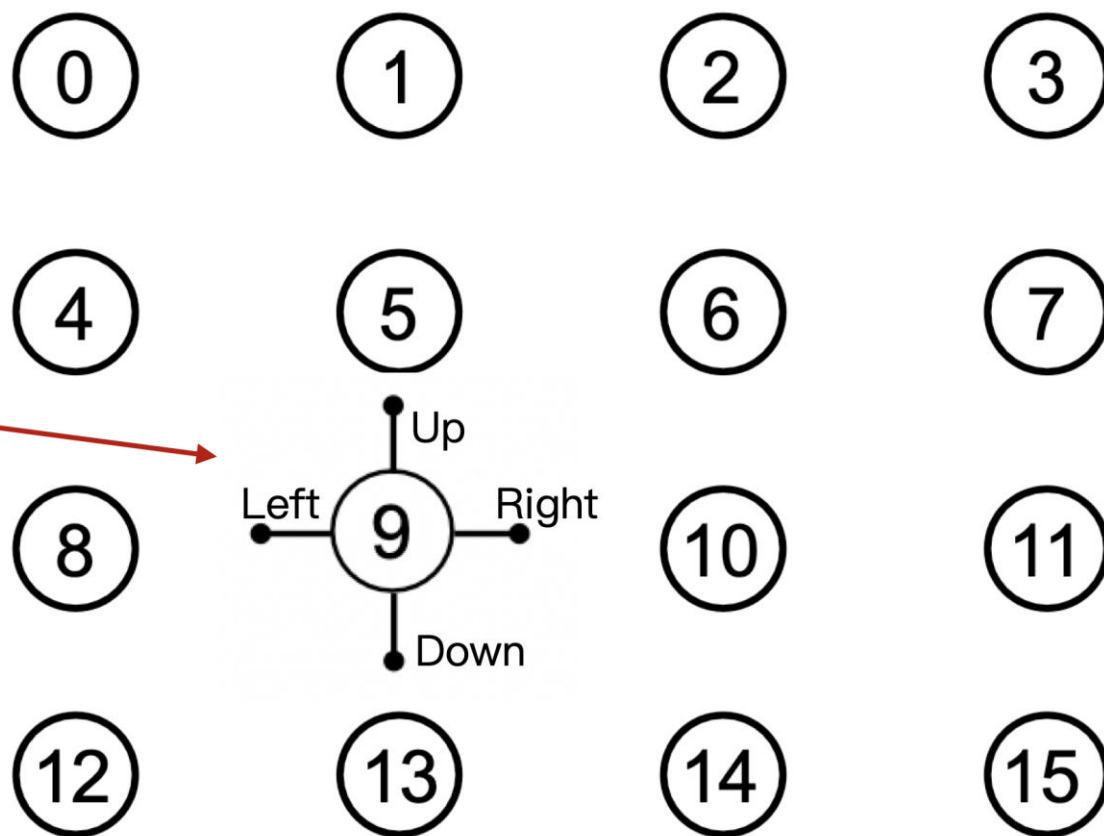
智能体怎么影响环境？

Action: A mechanism to influence the environment

↑ or ↓ or ← or →

Action space:

↑, ↓, ←, →



FL游戏案例

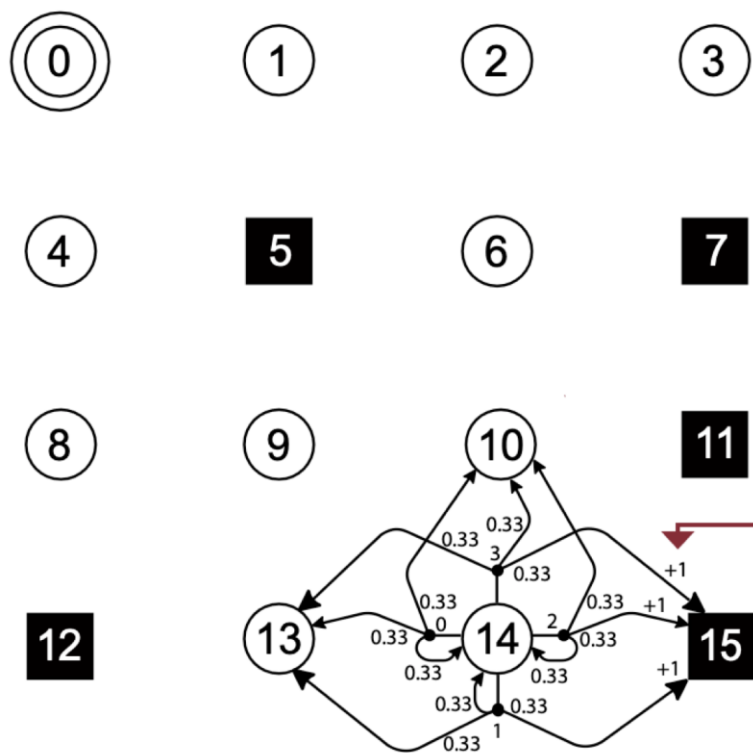
怎么评价智能体动作的即时得失？

稀疏 + 延迟

Reward: 只有到达位置15才会获得一个+1的奖励

Reward function:

$$r(s', s, a) = \mathbb{E}[R_t | S_{t-1} = s, A_{t-1} = a, S_t = s'], \text{ where } R_t \in R$$



Reward signal:
Carrot and stick

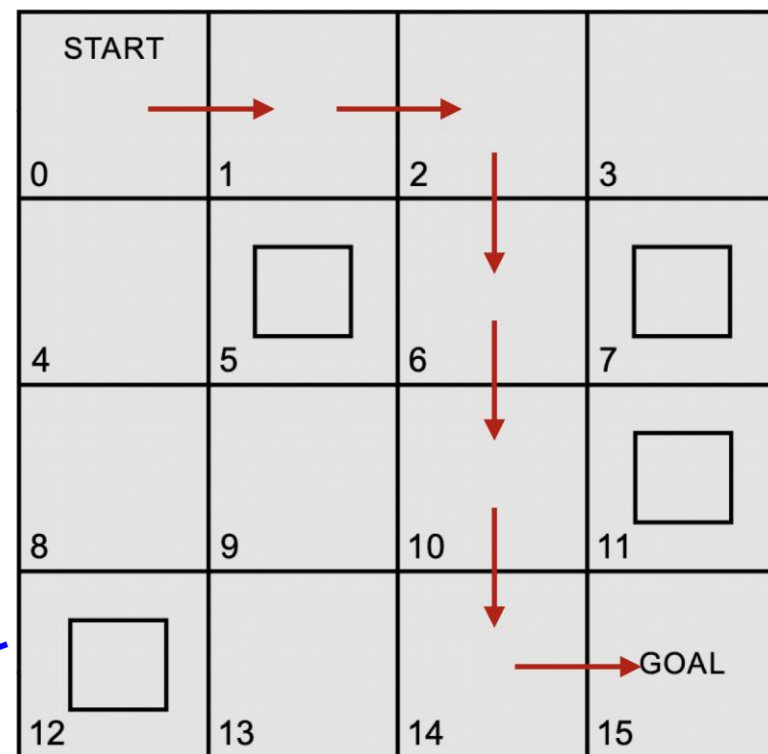


FL游戏案例

怎么评价智能体动作的即时得失？

- 回报 / Return: 从当前状态开始, 执行一系列动作, 在这个过程中获得的奖励的总和。对于回合 / episodic 任务, 通常是从当前状态到某一终态 / terminal state or absorbing state的过程中**获得的奖励总和**

- $G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$
- 通常采用折扣回报 / discounted return:
- $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-1} R_T$
- 其中 γ 是折扣因子



$$G_0 = 0 + 0 + 0 + 0 + 0 + 1 = 1 \quad | \quad G_0 = 0 + 0.9 * 0 + 0.9^2 * 0 + 0.9^3 * 0 + 0.9^4 * 0 + 0.9^5 * 1 = 0.59$$

FL游戏案例

怎么评价智能体动作的即时得失？

思考：为何引入折扣因子？

1. Mathematically convenient

$$100 + 100 + 100 + \dots + 100 = \infty$$

2. Future value of reward / money

我今天给你100元和我保证十年后给你100元，你选哪个？

3. Uncertainty in the environment

Day 1: 天气预报说明天会下雨，于是带上雨具是明智的。

Day 2: 大晴天，带雨具有点儿傻。

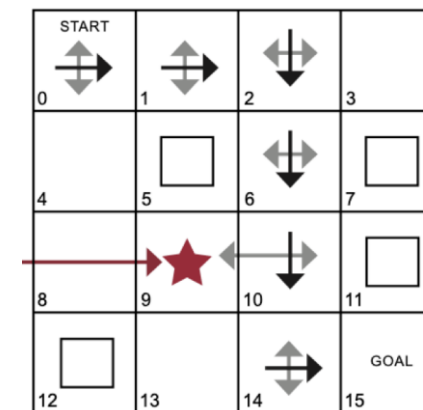
**Future is uncertain,
value it less!**

FL游戏案例

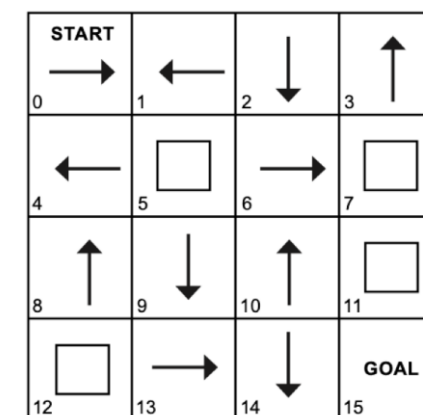
在每个状态下智能体应该选择什么动作？

- 策略 / Policy: 从状态到动作的映射/函数
 - 确定性策略: $a = \pi(s)$
 - 不确定/随机策略: $\pi(a|s)$
- 1. 策略的**必要性**? 我们做个规划plan不就好了?
 - 参考右图, 尽管我们有一个完美的规划, 但是这不一定能指导Agent的动作。比如在状态10的是选择向下走, 万一下一个状态是到了9怎么办? 状态9没有规划好的动作!
 - 策略定义了**从每个状态到相应动作的映射**
- 2. 策略的目的?
 - 策略的宗旨是**最大化从当前状态开始收集到的累计奖励**

只有计划不够

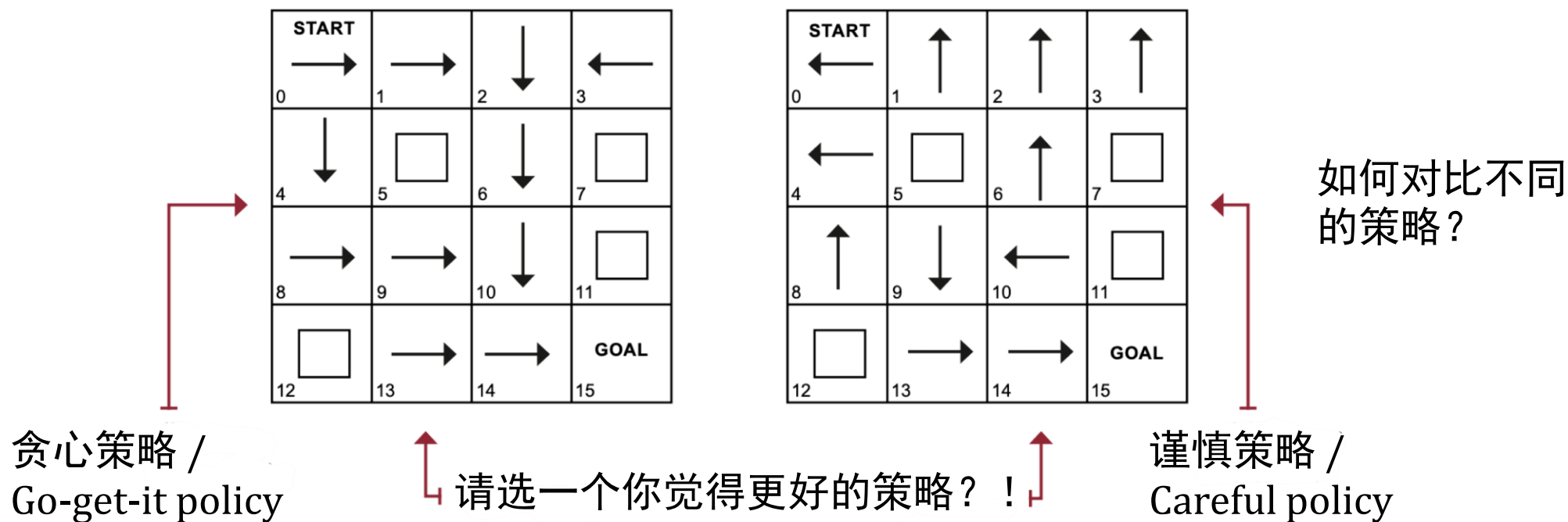


一个随机生成的策略



FL游戏案例

怎么找到最佳策略 — Maybe through exhaustive comparison?



- The whole point of RL is to **find the best policy** that maximizes the expected return.

FL游戏案例

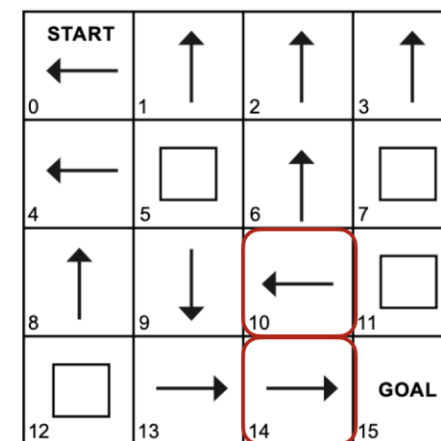
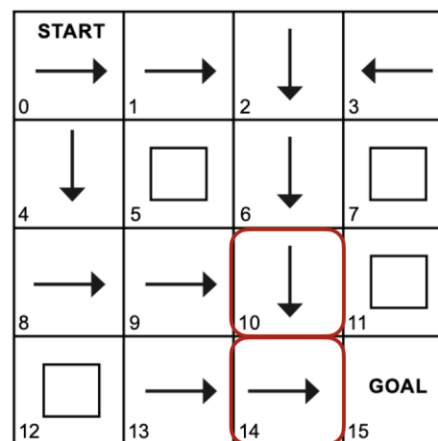
怎么对比/评估不同的策略 — **状态值函数**: what to expect from here?

- 值 / Value: 对Agent所处的状态好坏 / goodness的评估, 具体由值函数体现
 - 状态值函数 / State-value function: 给定状态s, 在此之后以策略选择动作执行, 获得的期望回报

$$v_{\pi}(s) = \mathbb{E}_{\pi}(G_t | S_t = s) = \mathbb{E}_{\pi}(R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots, + \gamma^{T-1} R_T | S_t = s)$$

How **good** is it to be in state s given that you follow policy π .

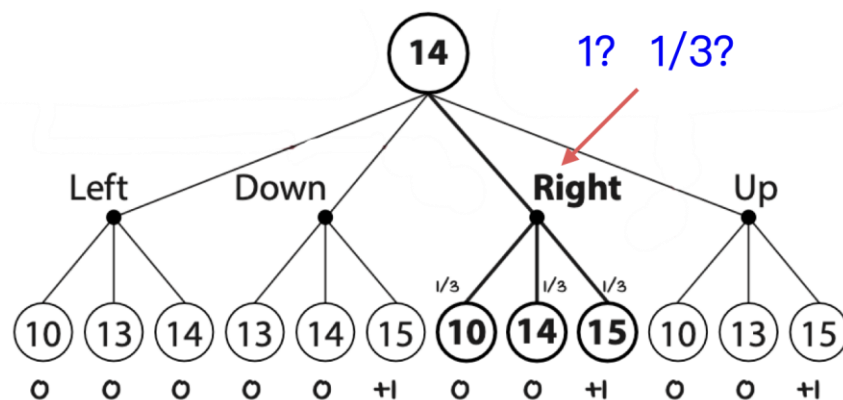
处在状态10和状态14相比, 哪个位置更好? 到底好多少? 是否和策略相关?



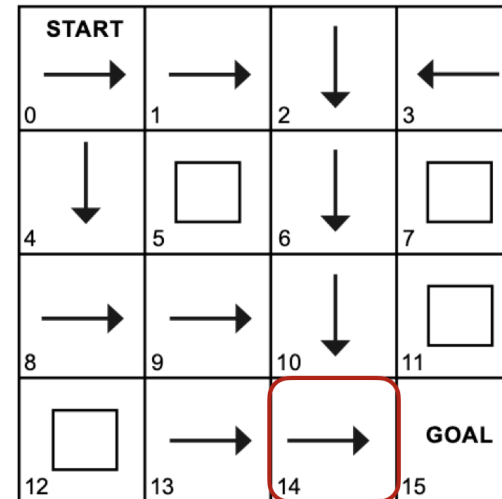
FL游戏案例

怎么评估给定策略 — 计算状态值函数？

- 状态值函数 $v_{\pi}(s) = \mathbb{E}_{\pi}(G_t | S_t = s) = \mathbb{E}_{\pi}(R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-1} R_T | S_t = s)$
- 贝尔曼方程: $v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + v'_{\pi}(s)], \forall s \in S$



在给定Policy的情况下，智能体处于14的位置到底有多好？



贝尔曼方程

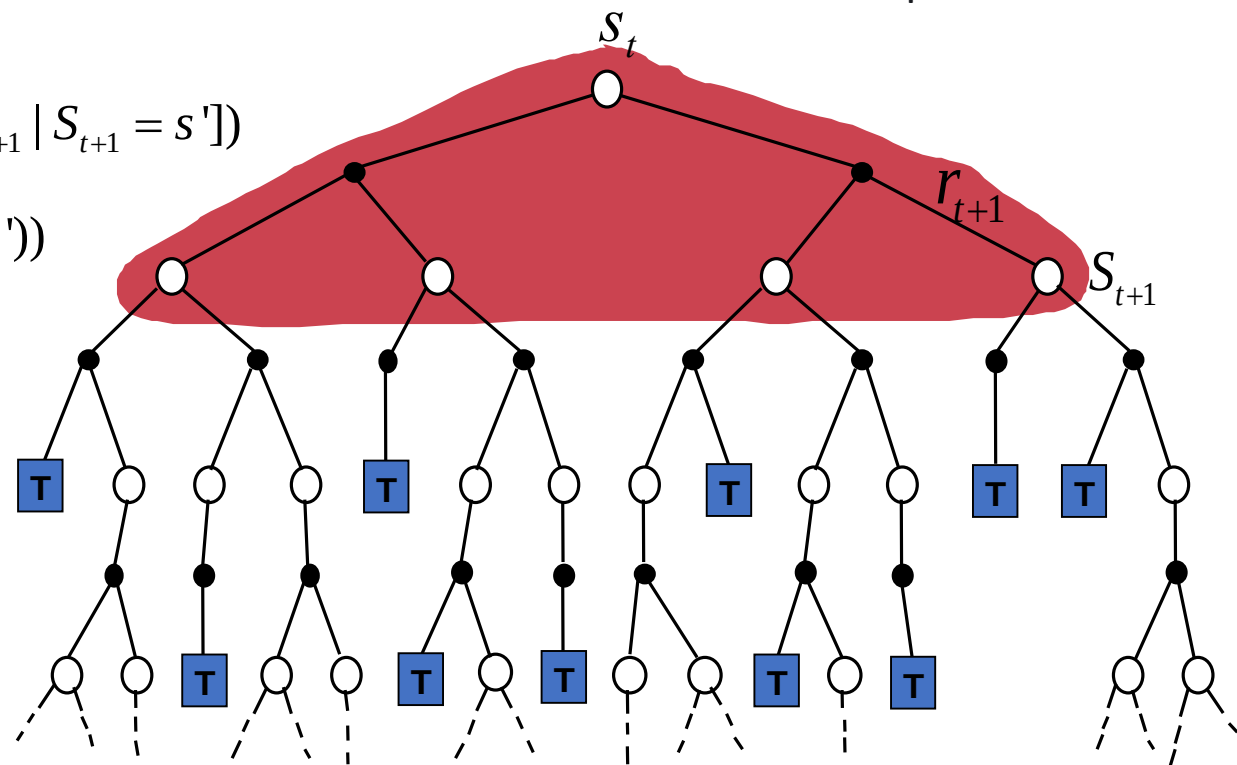
- 动态规划 (Dynamic Programming, DP) 是一种解决复杂问题的方法，它通过定义问题状态和状态之间的关系，将复杂问题拆分成若干较为简单的子问题，使得问题能够以递推（或者说分治）的方式去解决。所以要能使用动态规划，这种问题一要能够分解成许多子问题，二要这些子问题能够多次被迭代使用。而马尔科夫决策过程正好满足了这两个条件，MDPs可以看成是各个状态之间的转移，而贝尔曼方程则将这个问题分解成了一个**状态的递归求解问题**，而**值函数**就用于**存储这个求解的结果**，**得到每一个状态的最优策略**，合在一起**以后就完成了整个MDPs的求解**。但是DP的使用时建立在我们知道MDP环境的模型的基础上的，所以也称其为model based method。

贝尔曼方程推导

$$\begin{aligned}
 V^\pi(s) &= E[R_t | S_t = s] \\
 &= E[r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} \cdots) | S_t = s] \\
 &= E[r_{t+1} + \gamma R_{t+1} | S_t = s] \\
 &= \sum_{a \in A} \pi(s, a) \sum_{s' \in S} p(s' | s, a) (R(s' | s, a) + \gamma E[R_{t+1} | S_{t+1} = s']) \\
 &= \sum_{a \in A} \pi(s, a) \sum_{s' \in S} p(s' | s, a) (R(s' | s, a) + \gamma V^\pi(s')) \\
 &= E[r_{t+1} + \gamma V^\pi(s') | S_t = s]
 \end{aligned}$$

- 第四步是动作-状态全概率展开，相当于写出当前状态 s 到下一个所有可能的状态 s' 的转换概率，再根据转换概率求和。

这幅图叫做backup diagrams（备份图），其中每一个空心圆圈代表一个状态（state），一个实心的圆圈代表一个状态-动作对（state-action pair）



FL游戏案例

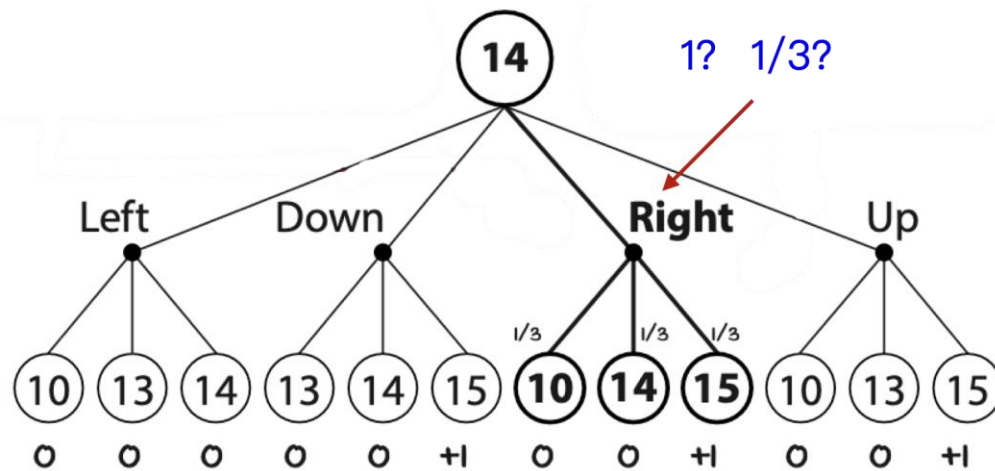
怎么评估给定策略 — 计算状态值函数？

- 贝尔曼方程:
$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + v'_{\pi}(s)], \forall s \in S$$

对不同动作产生的奖励加权

对不同的下一个状态和获得的奖励加权

对以不同概率获取的奖励以及下一个状态对应的值函数的折扣值加权



backup diagram (备份图)

FL游戏案例

怎么评估给定策略 — 计算状态值函数？

- 贝尔曼方程：

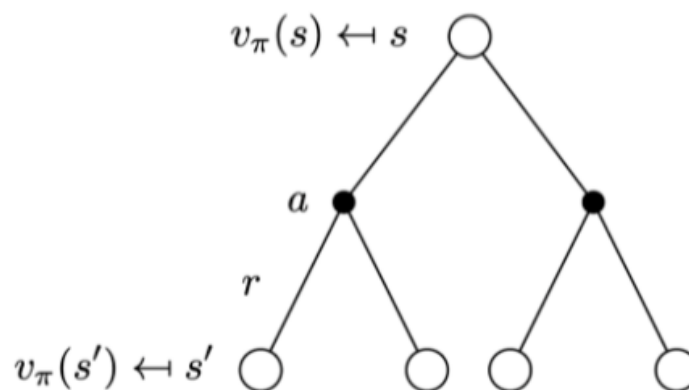
$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + v'_{\pi}(s)], \forall s \in S$$

状态 s' 的值函数我也不知道。这里我们会使用高斯-赛德尔迭代算法来求解，首先人为给一个初值，再根据下面的式子迭代求解，可以证明，当 k 趋于无穷时，最后是会收敛到 $v_{\pi}(s)$ 的。

FL游戏案例

Backup Diagram(备份图), 图示的关系构成了**更新或备份操作**的基础, 而这些操作是强化学习方法的核心。

Value function 的备份图



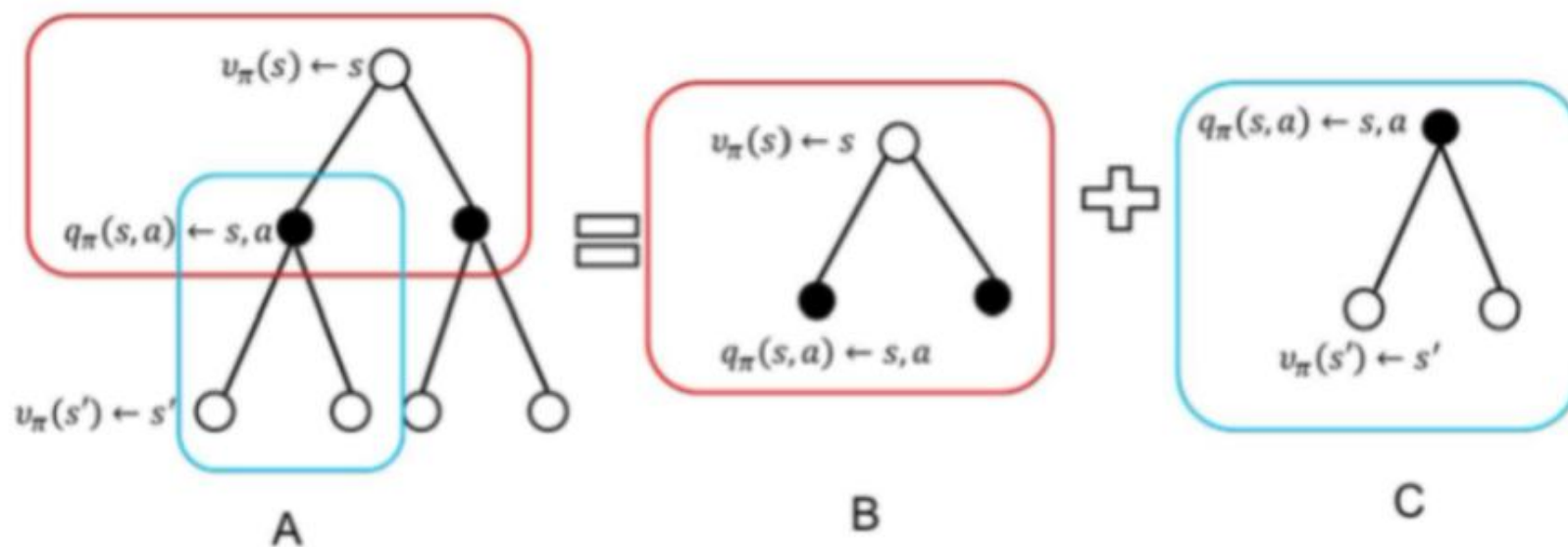
第一层加和就是这个叶子节点, 往上走一层的话, 我们就可以把未来的价值(状态s的价值) backup到黑色的节点。即: $q^\pi(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) v^\pi(s')$

第二层加和是对action进行加和。得到黑色节点的价值过后, 再往上backup一层, 就会推到根节点的价值, 即**当前状态的价值**。即: $v^\pi(s) = \sum_{a \in A} \pi(a | s) q^\pi(s, a)$

FL游戏案例

两者结合，有
$$v^\pi(s) = \sum_{a \in A} \pi(a | s) \left(R(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) v^\pi(s') \right)$$

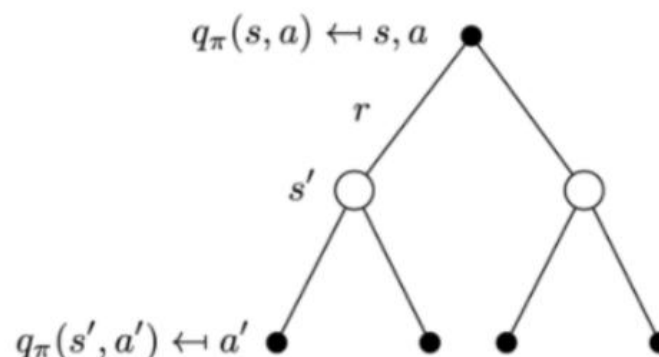
这个式子用Bellman Expectation Equation也可以推导出来



FL游戏案例

Backup Diagram(备份图), 图示的关系构成了**更新或备份操作**的基础, 而这些操作是强化学习方法的核心。

Q function 的备份图



$$q^\pi(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) \sum_{a' \in A} \pi(a'|s') q^\pi(s', a')$$

第一层加和是先把这个叶子节点从黑色节点推到这个白色的节点, 进了它的这个状态。即:

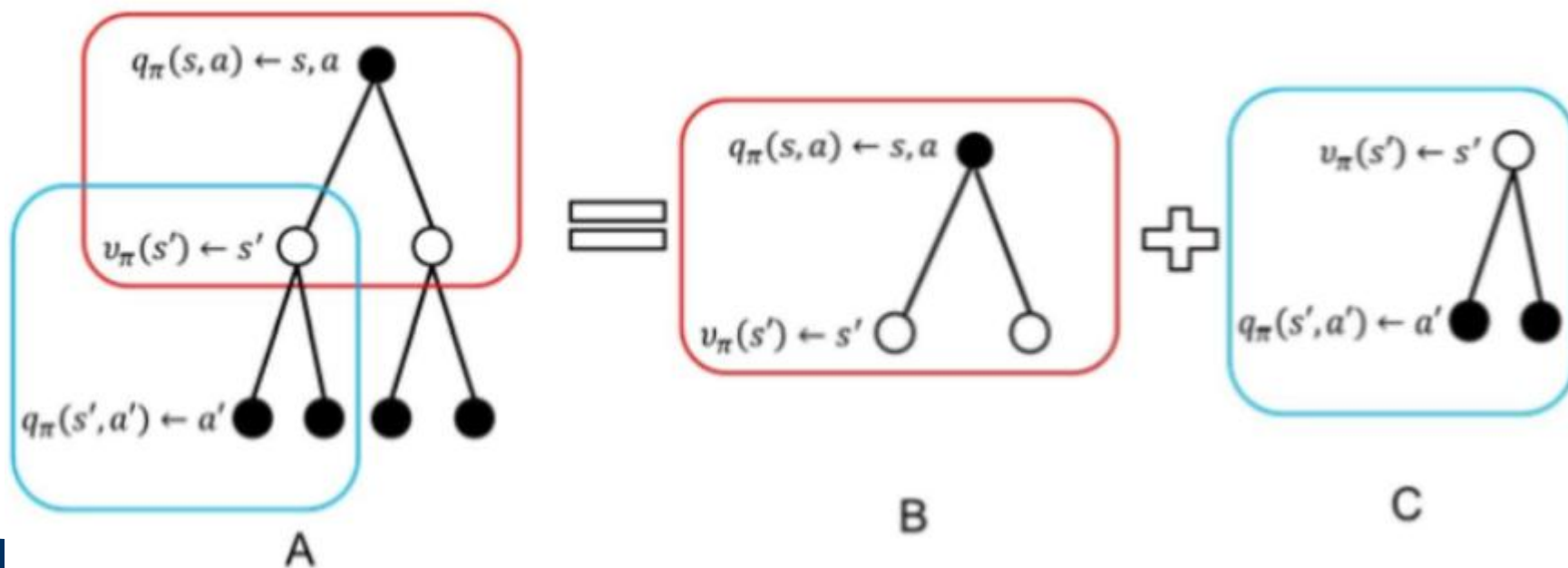
$$v^\pi(s') = \sum_{a' \in A} \pi(a' | s') q^\pi(s', a')$$

FL游戏案例

当我们到达某一个状态过后，再对这个白色节点进行一个加和，这样就把它重新推回到当前时刻的一个Q函数。即：

$$q^\pi(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) v^\pi(s')$$

两者结合，有

$$q^\pi(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) \sum_{a' \in A} \pi(a' | s') q^\pi(s', a')$$


FL游戏案例

怎么改进给定策略（策略提升） — 动作值函数: What should I expect from here if I do this?

- 动作值函数 / Action-value function: 在**状态s时先采取动作a**，然后再跟着给定的**策略**走的期望回报，通常叫做Q-value

$$\begin{aligned} q_{\pi}(s, a) &= \mathbb{E}[G_t | S_t = s, A_t = a] \\ &= \sum_{s', r} p(s', r | s, a)[r + \gamma v'_{\pi}(s)], \forall s \in S, \forall a \in A(s) \end{aligned}$$

What is the **quality (hence Q-value)** of the action a taken in state s ?

我们用一种贪婪的方式来提升我们策略，即选择那个能使**动作值函数最大的动作**：

$$\pi'(s) = \underset{a \in \mathcal{A}}{\operatorname{argmax}} q_{\pi}(s, a)$$

FL游戏案例

怎么改进给定策略（策略提升） — 动作值函数: What should I expect from here if I do this?

- 动作值函数 / Action-value function: 在状态 s 时先采取动作 a ，然后再跟着给定的策略走的期望回报，通常叫做Q-value

$$\begin{aligned} q_{\pi}(s, a) &= \mathbb{E}[G_t | S_t = s, A_t = a] \\ &= \sum_{s', r} p(s', r | s, a)[r + \gamma v'_{\pi}(s)], \forall s \in S, \forall a \in A(s) \end{aligned}$$

What is the quality (hence Q-value) of the action a taken in state s ?

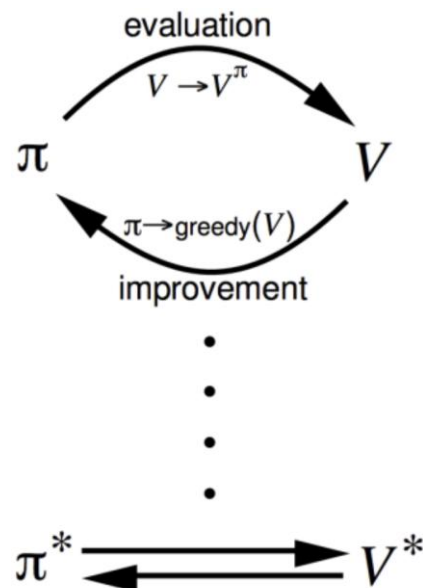
我们用一种贪婪的方式来提升我们策略，即选择那个能使动作值函数最大的动作：

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} q_{\pi}(s, a)$$

FL游戏案例

策略迭代 (Policy Iteration)

- 说完了策略评估和策略提升，策略迭代就简单了，就是反复使用策略评估和策略提升，最后会收敛到最优策略。



Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$
2. Policy Evaluation
 Loop:
 $\Delta \leftarrow 0$
 Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)
3. Policy Improvement
 $policy_stable \leftarrow true$
 For each $s \in \mathcal{S}$:
 $old_action \leftarrow \pi(s)$
 $\pi(s) \leftarrow \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$
 If $old_action \neq \pi(s)$, then $policy_stable \leftarrow false$
 If $policy_stable$, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

FL游戏案例

寻找最佳策略的路径 — 两种值函数的对比（贝尔曼方程Bellman Equation）

- 状态值函数：
 - 用来**评估**给定策略的好坏
 - 需联合马尔可夫决策过程（MDP）模型才能找到最佳策略
 - 解决**预测问题**
- 动作值函数：
 - 用来**改进**给定策略，
 - 不需要MDP就可以给出最佳策略
 - 解决**控制问题**

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v'_{\pi}(s)], \forall s \in S$$

$$q_{\pi}(s, a) = \sum_{s',r} p(s',r|s,a)[r + \gamma v'_{\pi}(s)], \forall s \in S, \forall a \in A(s)$$

- 前者是对环境中**某一个状态的价值大小**做的评估，后者是对在**某状态下的动作的价值大小**的评估。

FL游戏案例

寻找最佳策略的路径 — 两种值函数的对比（贝尔曼方程Bellman Equation）

函数	含义	关注重点
$v(s)$	在状态 s 下按照策略 π 的期望回报	只看状态
$q(s,a)$	在状态 s 下执行动作 a ，并之后遵循策略 π 的期望回报	同时看状态与动作

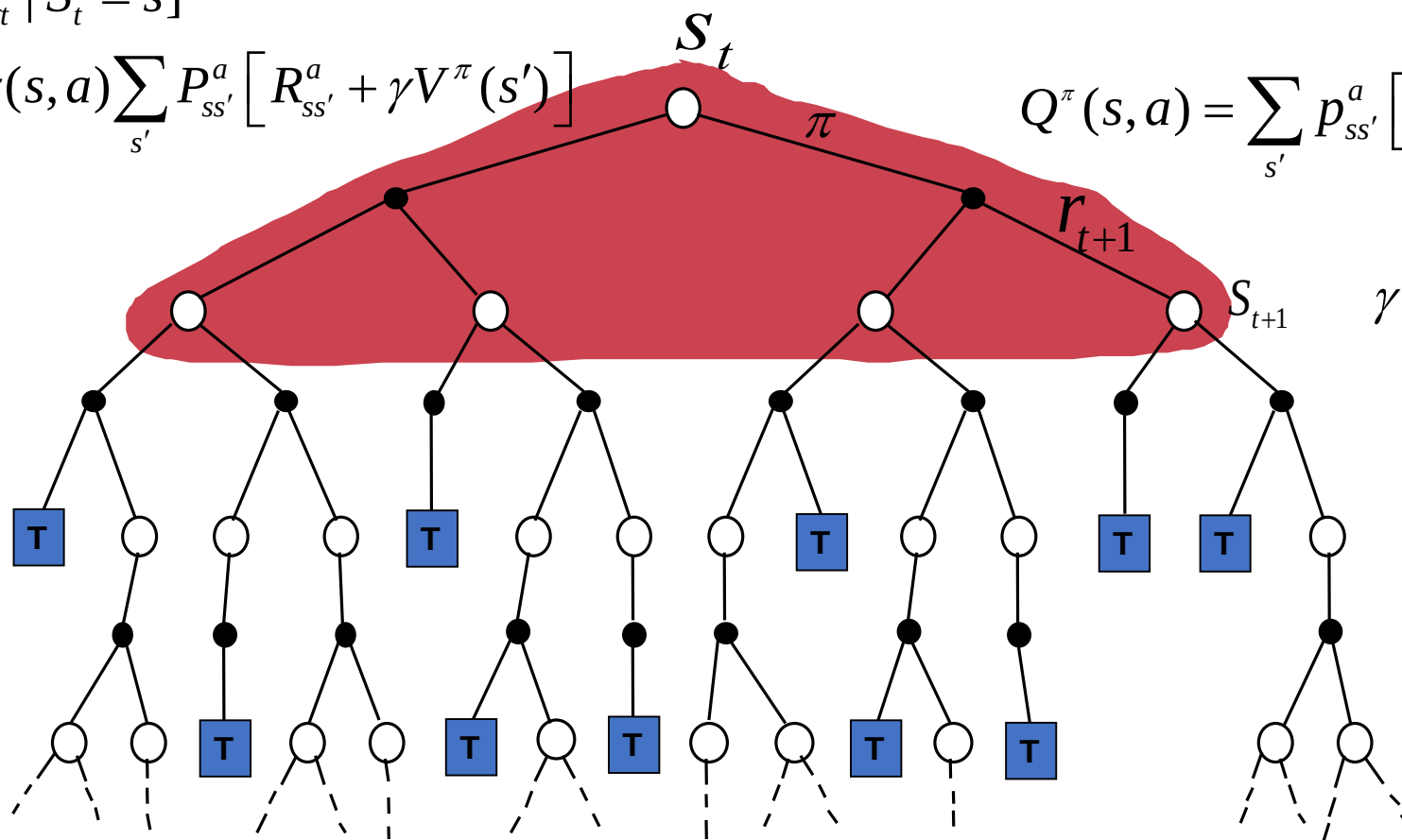
v 函数回答“**我在这里的价值**是多少”，而 q 函数回答“**我在这里做某件事的价值**是多少”。

动作-状态全概率展开

$$V^\pi(s) = E[R_t | S_t = s]$$

$$= \sum_a \pi(s, a) \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^\pi(s')]$$

$$Q^\pi(s, a) = \sum_{s'} p_{ss'}^a [R_{ss'}^a + \gamma V^\pi(s')]$$



$$\gamma E[R_{t+1} | S_{t+1} = s']$$

描述了当前状态值函数和其后续状态值函数之间的关系，即状态值函数（动作值函数）等于瞬时回报的期望加上下一状态的（折扣）状态值函数（动作值函数）的期望。

贝尔曼最优方程 (Bellman Optimality Equation)

强化学习的目标是找到一个最优策略，使得回报最大。准确的说是使值函数最大，包括状态值函数和动作值函数，分别记为

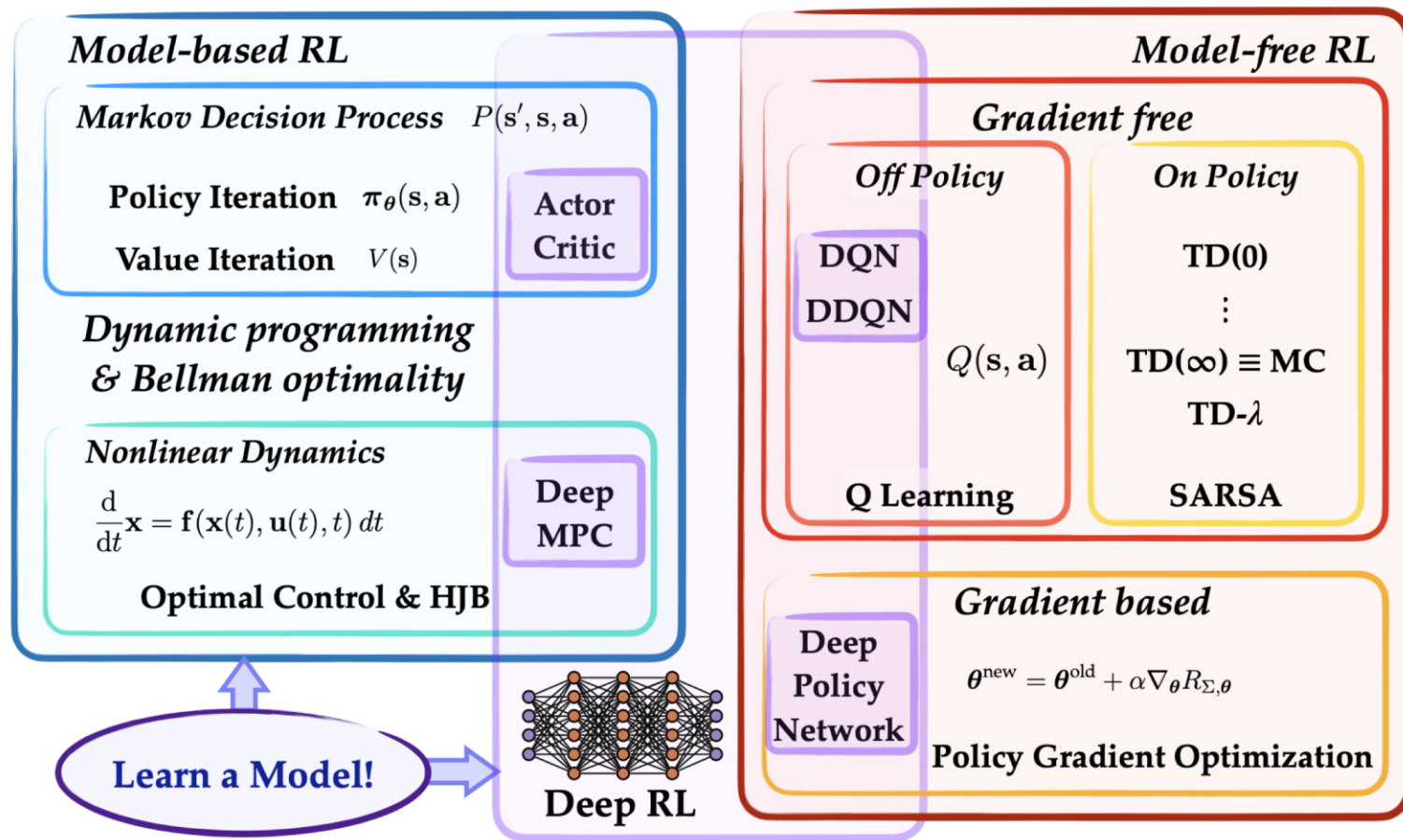
$$V^*(s) = \max_{\pi} V^{\pi}(s) \quad Q^*(s) = \max_{\pi} Q^{\pi}(s, a)$$

对于任意一个MDPs，总是存在一个最优的策略，在使用这个策略时就能取得最优值函数，即 $V^{\pi^*}(s) = V^*(s)$ ， $Q^{\pi^*}(s, a) = Q^*(s, a)$ ，于是我们可以得到贝尔曼最优方程

$$V^*(s) = \sum_a \pi(s, a) \sum_{s'} P_{ss'}^a \left[R_{ss'}^a + \gamma V^*(s') \right]$$
$$Q^*(s, a) = \sum_{a'} P_{ss'}^a \left[R_{ss'}^a + \gamma V^*(s', a') \right]$$

上述贝尔曼最优方程是非线性的，不存在闭式解，常常使用一些迭代的方法求解，比如值迭代、策略迭代、Q-learning、Sarsa等，这就是我们后面要学习的内容了。

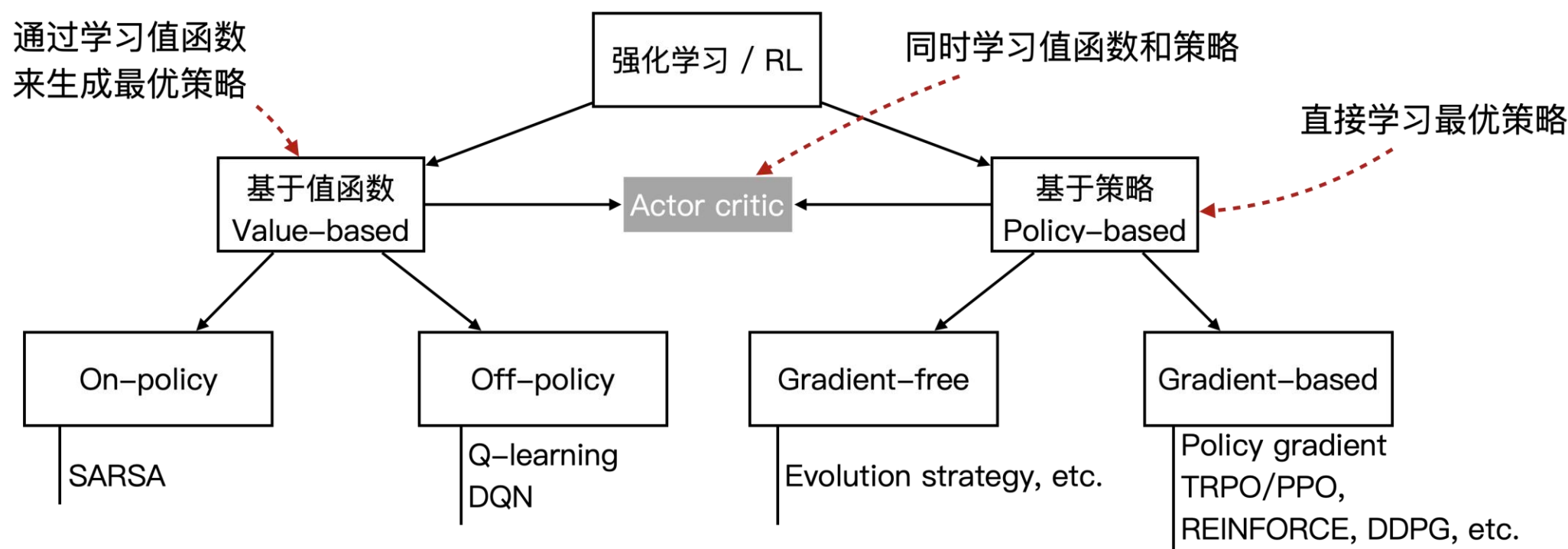
强化学习的分类



- 根据不同的准则可以将RL分为不同的类别。随着RL的发展不同类别以及分类方法的界限越来越不明确。

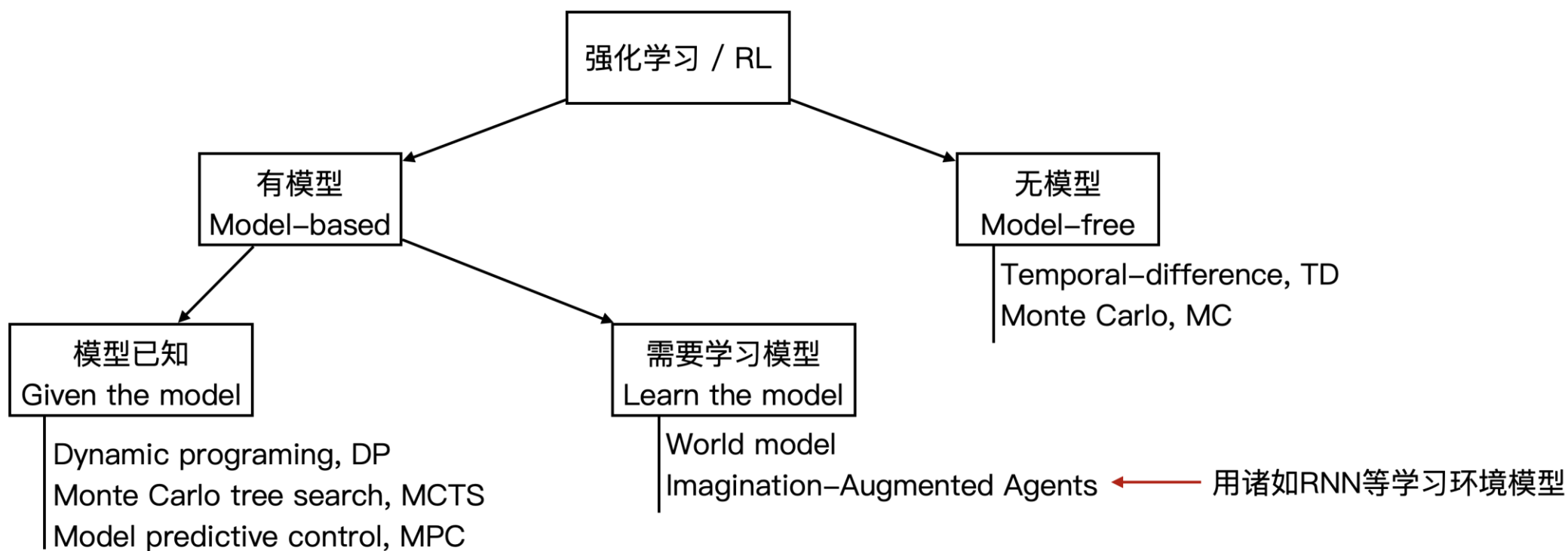
强化学习的分类

- 根据学习的对象不同：



强化学习的分类

- 根据是否需要环境模型 (recall this includes both transition function and reward function):



基于模型的RL

如何寻找最优策略？ — 首先得有一个办法评估给定策略的好坏 (1)

策略评估 / Iterative policy evaluation:

- 策略 π 比策略 μ 好，当且仅当在任意一个状态下跟随策略 π 得到的预期回报都比跟随策略 μ 得到的预期回报高

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v_k(s')]$$

$$v_{\pi}(s) > v_{\mu}(s)$$

k 是迭代次数

用来**对比**不同策略的好坏，因为是给定策略，所以是个**预测问题**

基于模型的RL

如何寻找最优策略？ — 其次得有个办法生成更好的策略

策略改进 / Policy improvement

$$\pi'(s) = \underset{a}{\operatorname{argmax}} \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$

- 利用状态值函数和问题的MDP模型来得到动作值函数
- 在每个状态上贪心选择更好的动作，从而生成一个新的策略

基于模型的RL

值迭代 (Value Iteration)

- 策略迭代有一个缺点，就是**每一步都要进行策略评估**，当状态空间很大的时候是非常耗费时间的。**值迭代是直接**将贝尔曼最优化方程拿来迭代计算的，这一点是不同于策略迭代的。
- 所以值迭代会直接收敛到最优值，从而我们就可以得到最优策略，因为它就是一个贪婪的选择。再反过去看一下策略迭代的过程，**策略评估过程是应用贝尔曼方程来计算当前最优策略下的值函数**，接着进行策略提升，即**在每个状态都选择一个最优动作来最大化值函数**，以改进策略。
- 在策略评估过程我们一定要等到它收敛到准确的价值函数吗？答案是不一定，我们可以设定一个误差，中断这个过程，用一个近似的值函数用以策略提升（格子世界的例子中就可以看出，在迭代到第三步以后，其实最优策略就已经确定了），而我们提出这个方法的时候并不是这么做的，而是等到策略评价过程收敛，这是一个极端的选择，相当于在迭代贝尔曼最优化方程！换句话说，**值迭代其实可以看成是策略迭代一个极端情况**。

基于模型的RL

能否找到比策略迭代更加快速的寻优方法？

值迭代 / Value iteration

数学上来说，值迭代中的操作可以合并成一个公式

$$v_{k+1}(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')]$$

- 一直利用以上公式进行迭代，直到收敛，最后再用计算动作值函数，用贪婪法来得到最佳策略
- 因为这个迭代过程不涉及策略本身，只涉及值函数，故名值迭代。
- 策略迭代和值迭代统称通用策略迭代 generalized policy iteration
- 策略迭代和值迭代的本质是在MDP模型下的动态规划算法

需要知道状态转移概率和奖励函数

基于模型的RL

- 在强化学习中，智能体在与环境交互的过程中，应该根据当前状态做出一个动作（action），并获得一个奖励（reward）。
- 然而，智能体并不仅仅关心立即获得的奖励，而是希望在长期内获得尽可能高的回报（return）。
- 为了实现这一目标，我们要求衡量在不同状态下采取不同动作的“好坏”。于是，状态-动作值函数（State-Action Value Function），也称为 Q 函数（Q-function），应运而生。
- Q 函数能告诉智能体：“在当前状态下执行哪个动作，未来能带来最大的长期收益。”

基于模型的RL

- 强化学习的目标是最大化长期累积回报（Return），而 Q 函数正是对这种长期回报的数学化刻画。
- Q 函数（State-Action Value Function）的定义如下：

$$Q(s, a) = \mathbb{E}[R_t \mid s_t = s, a_t = a]$$

- s_t ：表示在时间步 t 时刻的状态；
- a_t ：表示在状态 s_t 下采取的动作；
- R_t ：表示从时间步 t 开始的累计回报（Return）。

累计回报 R_t 可能写作： $R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots + \gamma^{T-1} r_T$

- 即时奖励（reward）；就是 r_t
- $\gamma \in [0, 1]$ 是折扣因子（discount factor），用于平衡短期和长期收益。

基于模型的RL-根据 Q 值选择最优动作

通过这些计算，我们可以发现：

Q 值不仅与状态 s 有关，还强烈依赖于动作 a 。

在相同状态下，不同动作的回报差距可能很大。

这也正是强化学习要解决的问题——**如何在每个状态下选择最优动作**。

既然我们已经计算出了每个状态下不同动作对应的 Q 值，那么问题就变成了：

智能体该如何根据这些 Q 值来选择下一步动作？

这就是**策略改进**（Policy Improvement）的核心思想。

(1) 动作选择原则

大家希望智能体在每个状态 s 下选择能带来最大长期回报的动作，也就是：
$$\pi(s) = \arg \max_a Q(s, a)$$

这意味着：对于每个状态 s ，我们计算所有可能动作的 $Q(s,a)$ ，然后**选择 Q 值最大的那个动作**。

基于模型的RL-根据 Q 值选择最优动作

(2) 最优 Q 函数 Q^*

我们定义：
$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$$

在所有可能的策略 π 中，某个状态-动作组合能够达到的最高期望回报。换句话说：

- $Q(s, a)$ 是在某个策略下的值；
- $Q^*(s, a)$ 是最优策略下的值。

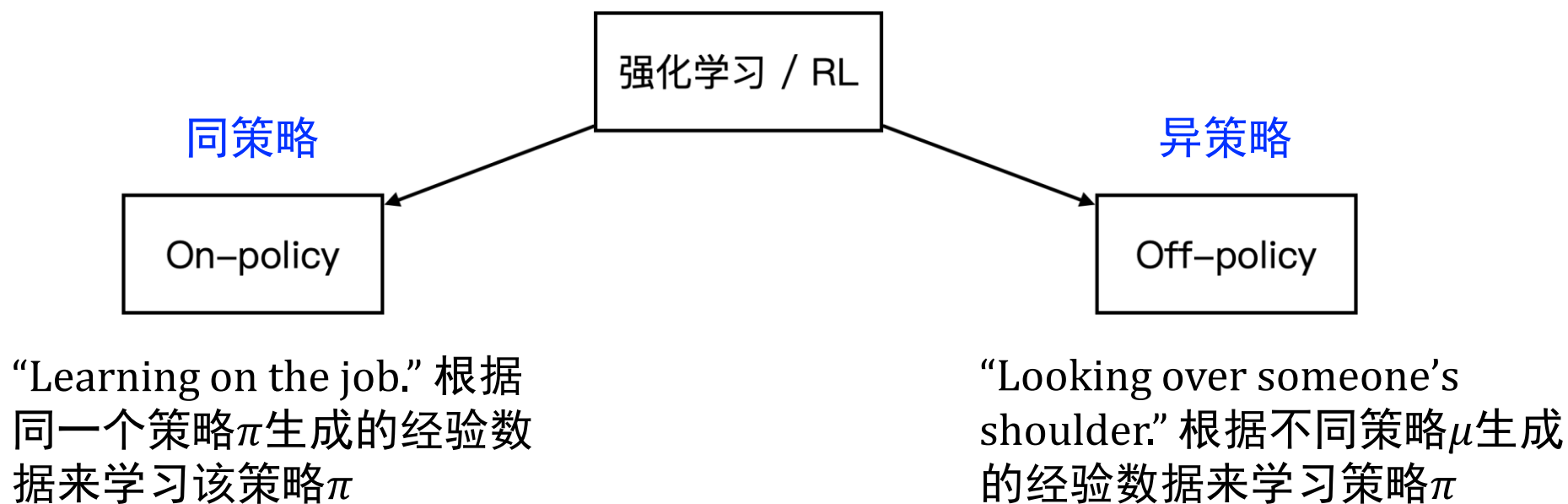
当我们得到了 Q^* 后，就可以轻松确定最优策略：
$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

- **Q 函数**：衡量状态-动作组合的长期回报。
- **最优 Q 函数 Q^*** ：描述在最优策略下每个动作的最终潜力。
- **最优策略 π^*** ：选择在每个状态中使 Q 值最大的动作。

强化学习的目标可能概括为：找到能最大化 $Q(s, a)$ 的策略 π 。

强化学习的分类

- 根据学习不同:



经典强化学习：Q学习

Q学习（其中一种Value-based RL方法，同类：SARSA）

- 无模型的（Model-free）
- 基于动作值函数 $Q(s, a)$
- 通过与环境交互直接训练 $Q(s, a)$ ，找到 $Q^*(s, a)$ 和 $\pi^*(s)$

Q-Learning的**目的**是学习特定State下、特定Action的价值。是建立一个Q-Table，以State为行、Action为列，通过每个动作带来的奖赏更新Q-Table。

即时奖励已知

状态

Q-table initialised at zero

	UP	DOWN	LEFT	RIGHT
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0
4	0	0	0	0
5	0	0	0	0
6	0	0	0	0
7	0	0	0	0
8	0	0	0	0

After few episodes

	UP	DOWN	LEFT	RIGHT
0	0	0	0	0
1	0	0	0	0
2	0	2.25	2.25	0
3	0	0	5	0
4	0	0	0	0
5	0	0	0	0
6	0	5	0	0
7	0	0	2.25	0
8	0	0	0	0

Eventually

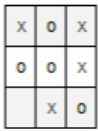
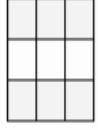
	UP	DOWN	LEFT	RIGHT
0	0	0	0.45	0
1	0	1.01	0	0
2	0	2.25	2.25	0
3	0	0	5	0
4	0	0	0	0
5	0	0	0	0
6	0	5	0	0
7	0	0	2.25	0
8	0	0	0	0

动作

Q值

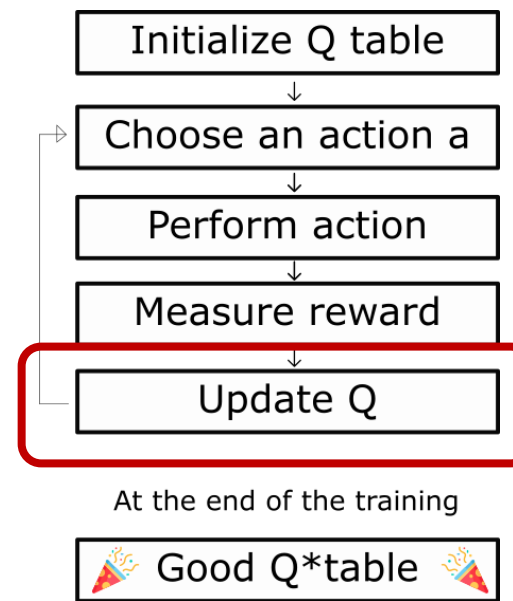
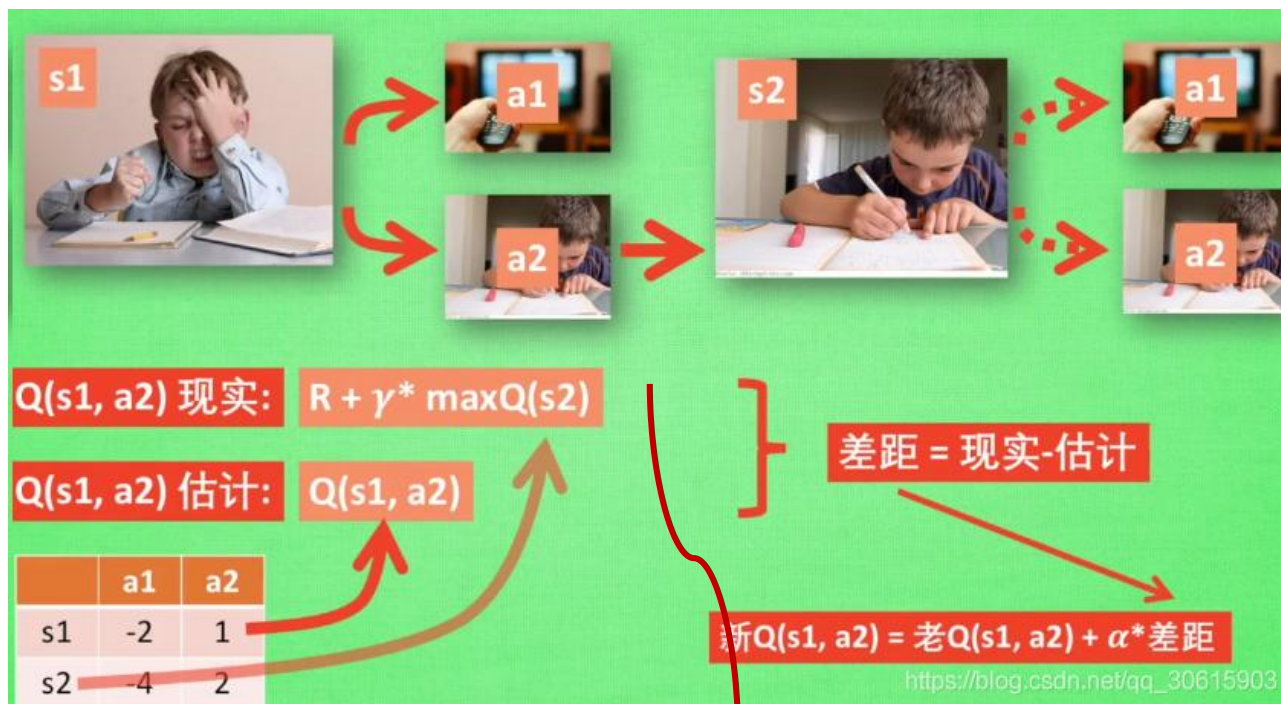
Q学习：Q表

- Q表的行：状态
- Q表的列：动作
- 单元格：状态动作对的Q值
- 可先假设状态数量已知
 - Q表可拓展：添加行和列

Game State	Top Left	Top Middle	Top Right	Middle Left	Middle Middle	Middle Right	Bottom Left	Bottom Middle	Bottom Right
	N/A	0.5	N/A	N/A	N/A	N/A	0	0	N/A
	N/A	N/A	N/A	N/A	N/A	N/A	0.5	N/A	N/A
	0.3	0.5	0.3	0.5	0.7	0.3	0.3	0.5	0.3
...	...	...	...	...	...	...	...	...	...



Q学习：更新Q值



$$newQ_{S,A} = Q_{S,A} + \alpha (R_{S,A} + \gamma * \max Q'(s', a') - Q_{S,A})$$

基于状态和
行动的新Q值

c当前Q值

学习效率

基于状态和
行动的奖励

折扣因子

在给定新的状
态和行动下未
来最大的奖励

Q学习与SARSA

- 一种使用时序差分求解强化学习控制问题的方法（此处不展开）

- 值迭代：

- 同策略学习：同一个策略选择动作和更新值函数，SARSA：

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$$

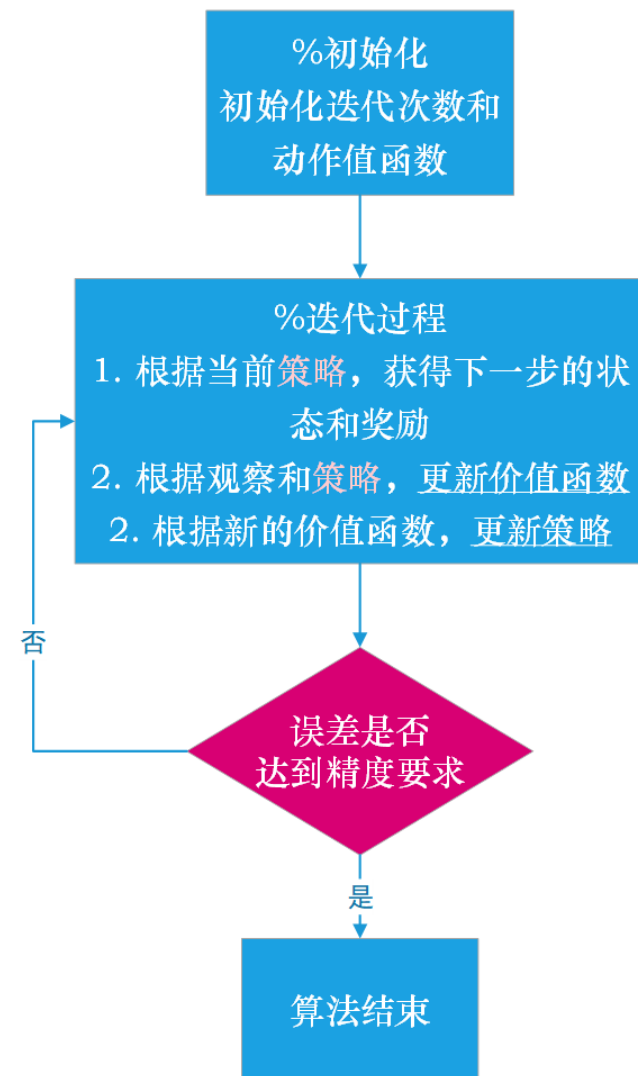
- 异策略学习：一个策略（我）选择动作，另一个策略（别人）更新值函数，Q学习：

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$$

SARSA是on-policy的更新方式，它的行动策略和评估策略都是ε-greedy策略

Q-Learning是off-policy的。异策略是指行动策略和评估策略不是一个策略。

Q-Learning中行动策略是ε-greedy策略，要更新Q表的策略是贪婪策略。



Q学习与SARSA

Q-Learning 算法：离线策略，学习的是**最优策略的Q值**（即假设下一步采取当前认为的最优动作，无论实际策略如何选择动作）。

SARSA 算法：在线策略，学习的是**当前策略的Q值**（即下一步实际采取的动作 a_{t+1} 由当前策略生成）。

维度	Q-Learning	SARSA
策略类型	离线策略（学习最优策略）	在线策略（学习当前策略）
目标Q值	下一状态的最优动作值 ($\max Q(s', a')$)	下一状态的实际动作值 ($Q(s', a')$)
探索方式	基于最优动作的估计（可能过度乐观）	基于当前策略的动作（更谨慎）
收敛目标	最优Q值 (Q^*)	当前策略的Q值 (Q^π)
样本效率	较低（依赖最优估计的偏差）	较高（样本与策略一致）
风险偏好	更冒险（追求潜在最优）	更保守（避免未知风险）

Q学习与SARSA

Q-Learning: 通过“想象”最优动作来更新，适合探索未知环境并直接优化最优策略。

SARSA: 通过“执行”当前策略的动作来更新，适合需要策略一致性的场景（如安全敏感任务）。

- Q-Learning: “看着最好的路走”（离线最优）。
- SARSA: “沿着自己的路走”（在线策略）。

假设状态 s ，动作 a ，奖励 r ，下一状态 s' ：

算法	动作选择（当前策略）	更新目标（下一状态Q值）	公式简写
Q-Learning	ϵ -贪心	$\max_a Q(s', a)$	$Q(s, a) \leftarrow r + \gamma \max_a Q' - Q$
SARSA	ϵ -贪心	$Q(s', a')$ （实际选择的 a' ）	$Q(s, a) \leftarrow r + \gamma Q' - Q$

值函数的近似表示

- 如果问题的状态集合规模大，Q表的维护和查询困难

- 建立值函数的近似表示

- 引入一个状态值函数 $\hat{V}(s, \theta)$ ，由参数 θ 描述，接受状态 s 作为输入，计算后得到状态 s 的值函数：

$$\hat{V}(s, \theta) \approx V_{\pi}(s)$$

- 类似地，引入动作值函数 $\hat{Q}(s, a, \theta)$ ：

$$\hat{Q}(s, a, \theta) \approx Q_{\pi}(s, a)$$

- 函数近似的方法：

- 线性拟合，三次拟合，决策树，伪谱法...
- 神经网络，深度神经网络